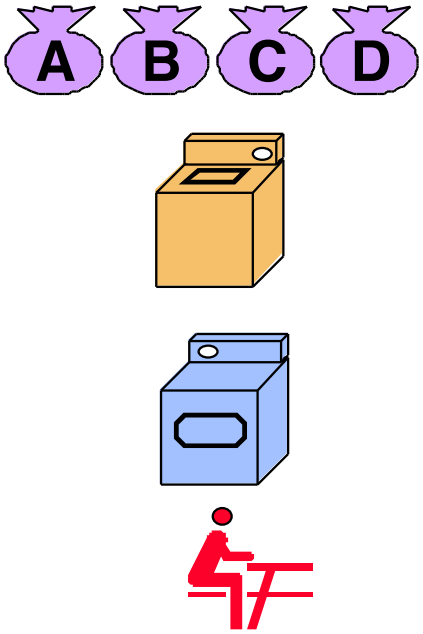
Advanced Computing Architectures

3 – Pipelining

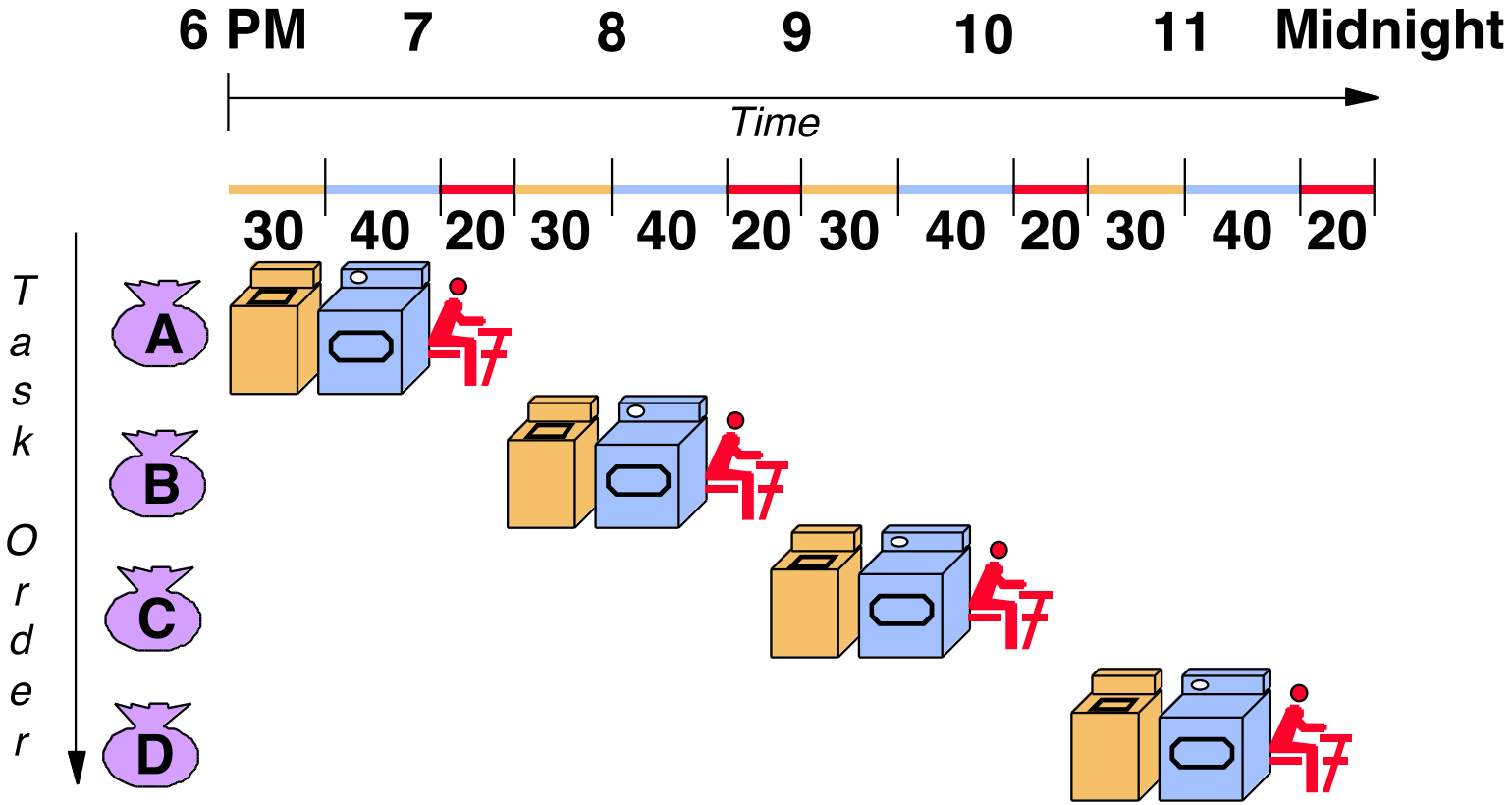
- Definition and advantages of a pipeline
- Non-pipelined and pipelined implementation of a RISC processor
- Dependences and hazards
  - Structural Hazards
  - Data Hazards
  - Control Hazards
- Exception handling

Pipelining: Its Natural!

- Laundry Example
- Ann, Brian, Cathy, Dave each have one load of clothes to wash, dry, and fold
- Washer takes 30 minutes
- Dryer takes 40 minutes
- "Folder" takes 20 minutes

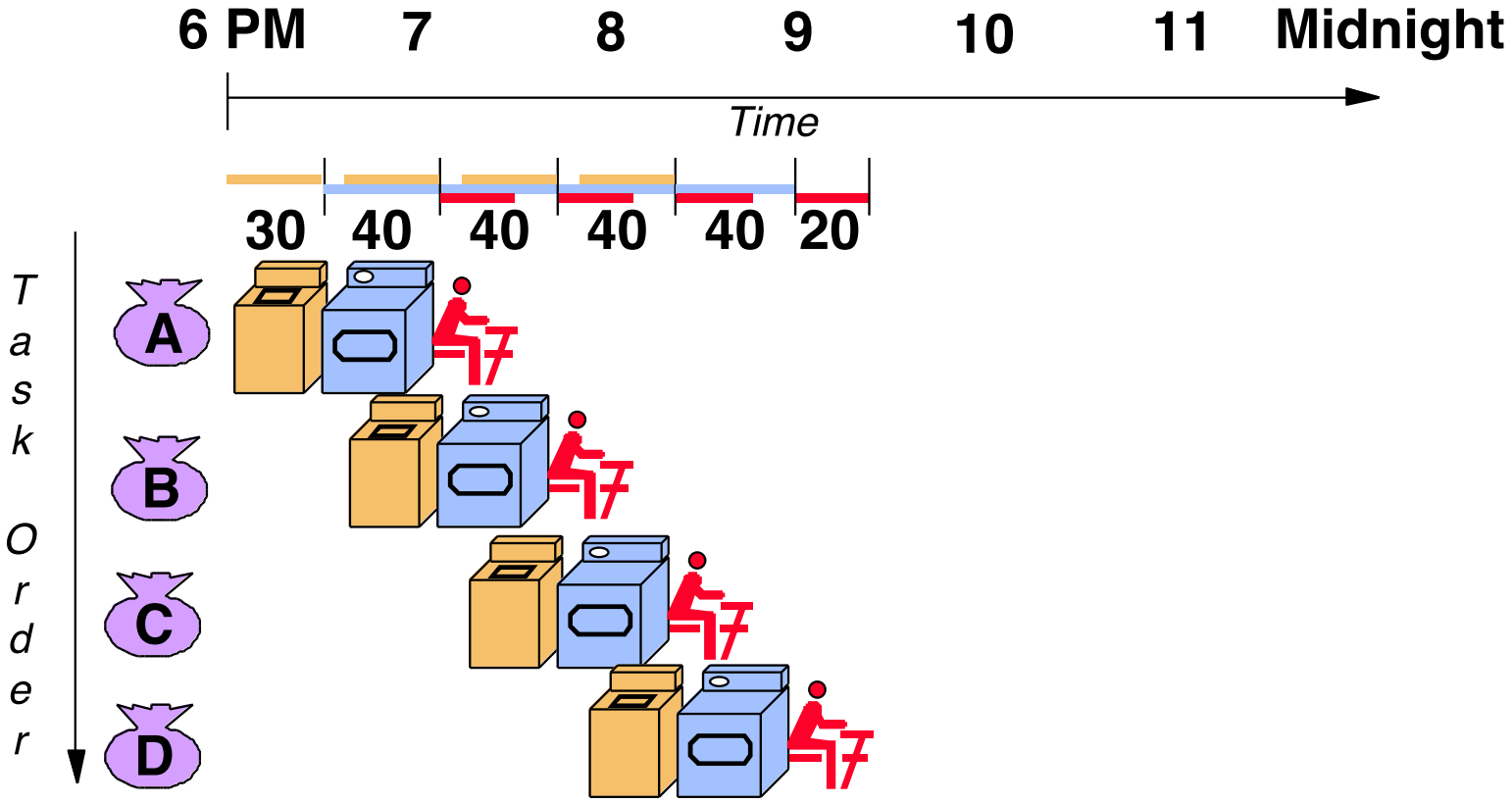


Sequential Laundry



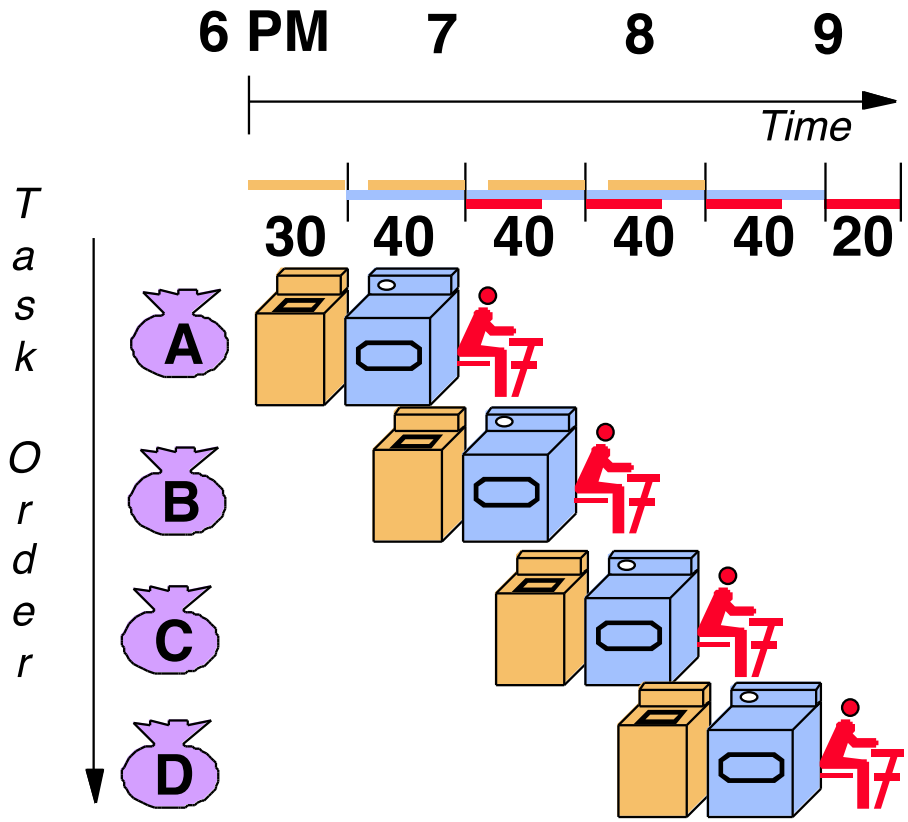
- Sequential laundry takes 6 hours for 4 loads
- If they learned pipelining, how long would laundry take?

Pipelined Laundry
Start work ASAP



- **Pipelined laundry takes 3.5 hours for 4 loads**

Pipelining Lessons



- Pipelining doesn't help latency of single task, it helps throughput of entire workload
- Pipeline rate limited by slowest pipeline stage
- Multiple tasks operating simultaneously
- Potential speedup = Number pipe stages
- Unbalanced lengths of pipe stages reduces speedup
- Time to "fill" pipeline and time to "drain" it reduces speedup

What is a pipeline ?

- The basic idea of *Pipelining* is to **divide the elementary operations related to a single instruction into a sequence of smaller tasks that can be performed in parallel**
- In this way, at the same time elementary tasks of subsequent instructions can be executed in different pipeline *segments* or *stages*
- The goal is to balance the length of each pipeline stage. This way, the average time per instruction (pipelined) when the pipeline is full is:

Time per instruction (unpipelined) / Number of stages

- Pipelining reduces the average CPI.
  - **Example**: If each instruction in a microprocessor takes 5 clock cycles (unpipelined) and we have a 4-stage pipeline, the ideal average CPI with the pipeline will be 1.25

Ideal pipeline performance

- If the pipeline consists of p stages and if T_i is the total latency of a single instruction, the delay of an individual stage is

$$t_s = t_o + T_i / p,$$

t_o being the overhead latency of each stage

- CPI
  - Each instruction takes the same number of clocks in the pipelined and in the unpipelined case (latency of instructions does not change)
  - But the **execution clocks are shared**
  - The CPI is the number of clocks required (on average) to complete one instruction after the previous one has completed
- Hence
  - In a pipeline, **the throughput** of any instruction sequence (i.e., the number of completed instruction per unit of time) **is about *p* times larger** than in the unpipelined solution

Implementing a basic RISC CPU

- Pipeline structure and implementation depends on the ISA and on the number of functional units present in a certain processor
- We will consider the pipeline implementation of a typical RISC processor. Nevertheless, the basic concepts can be extended to more involved microarchitectures.

To simplify the analysis, we will assume that:

- Instructions are few in number and one size only (32 bit):
  - ALU instructions (direct register, immediate)
  - Load and store instructions (indirect register, displacement)
  - Branches (only based on comparison with 0) and jumps (PC-relative, indirect register)
- All ALU instructions apply to data in registers and typically change the entire register (e.g., 32-bits or 64-bits per register)
- The only instructions affecting memory are load/store

MIPS features

- 32 64-bit general purpose register (GPR), named R0, R1, …, R31. (R0 = 0)
- 32 floating-point registers (FPR), named F0, F1, …, F31.
- Data types: 8-bit bytes, 16-bit half words, 32-bit words, 64-bit double words.
- Addressing modes:
  - Immediate: ADD R4, R2, #3
  - Displacement: LD R4, 100(R1)
  - Register indirect: LD R4, 0(R1)
  - Absolute addressing: LD R4, 2536(R0)
- Load/store architecture

ISA instruction format

I-type instruction

6	5	5	16
Opcode	rs	rt	Immediate

Encodes: Loads and stores of bytes, half words, words, double words. All immediates (rt ← rs op immediate)

Conditional branch instructions (rs is register, rd unused)
Jump register, jump and link register
(rd = 0, rs = destination, immediate = 0)

R-type instruction

6	5	5	5	5	6
Opcode	rs	rt	rd	shamt	funct

Register-register ALU operations: rd ← rs funct rt
Function encodes the data path operation: Add, Sub, . . .
Read/write special registers and moves

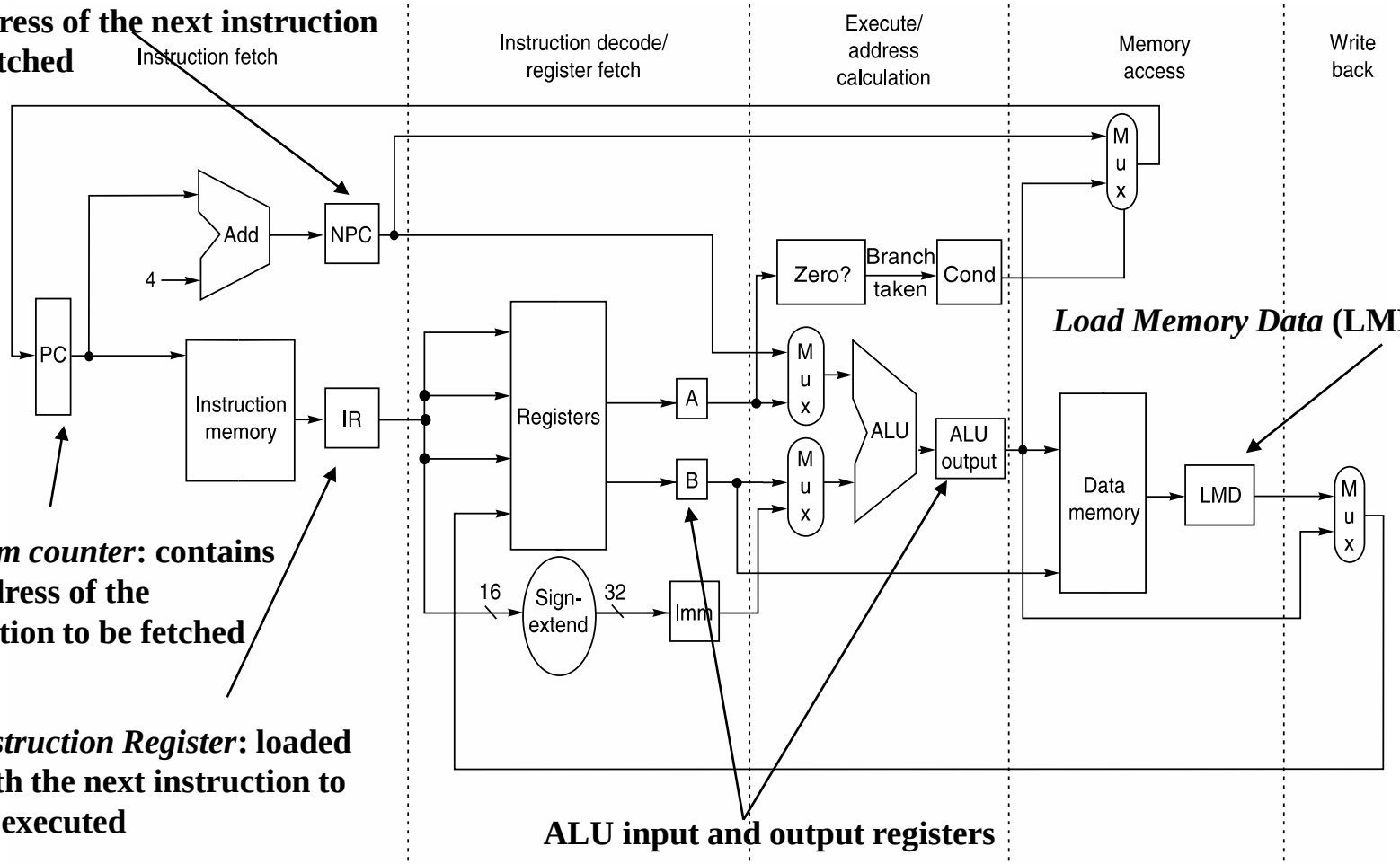
J-type instruction

6	26
Opcode	Offset added to PC

Jump and jump and link
Trap and return from exception

A non-pipelined RISC architecture

***New program counter*: contains the address of the next instruction to be fetched**



***Load Memory Data* (LMD) register**

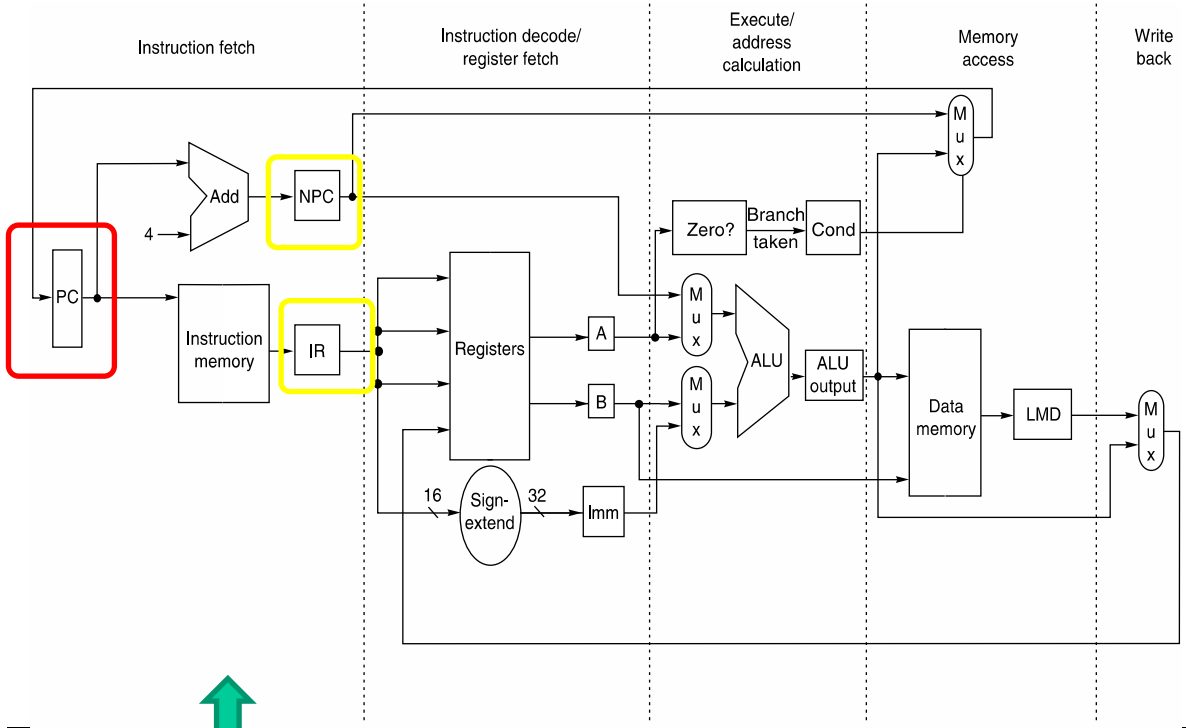
***Program counter*: contains the address of the instruction to be fetched**

***Instruction Register*: loaded with the next instruction to be executed**

ALU input and output registers

Instruction execution stages

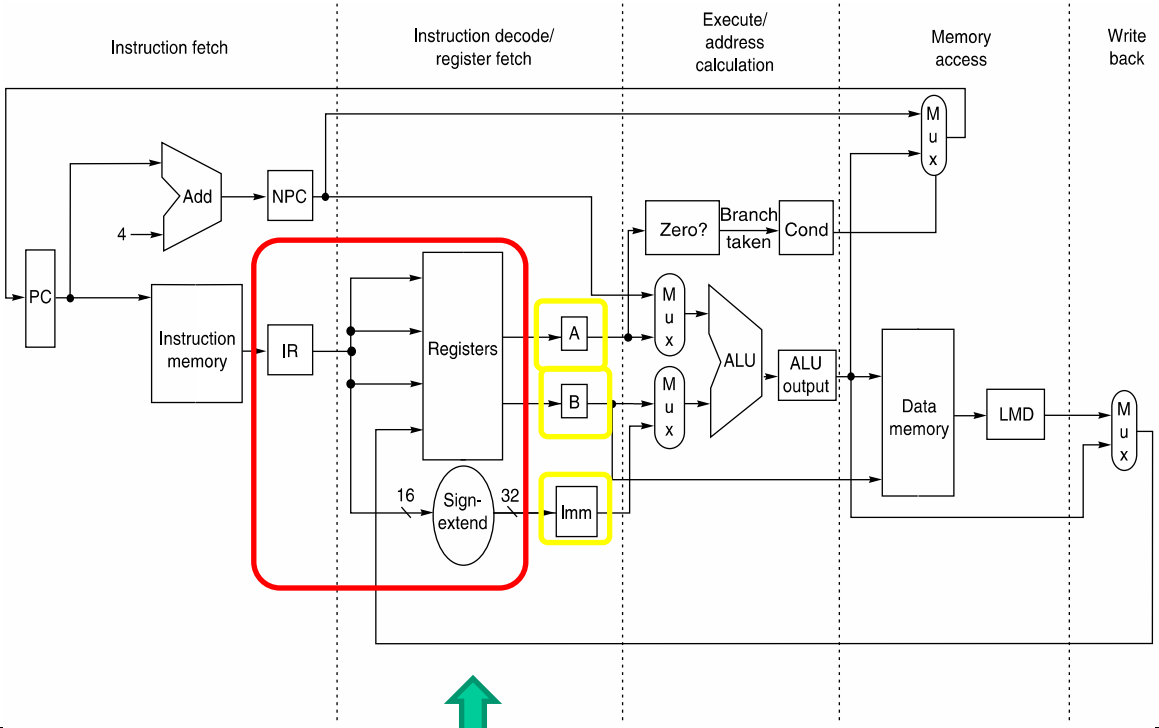
- Every instruction of the proposed RISC architecture can be implemented in at most **5 clock cycles**. The 5 stages are:
  - ***Instruction fetch stage* (IF)**: Send the program counter (PC) to memory and fetch the current instruction from memory or I-cache. Update the PC to next sequential value (e.g., by adding 4 if an instruction is 4 byte long)



Instruction execution stages

– ***Instruction Decode/ register fetch stage* (ID):** decode the instruction and perform simultaneous operand reading. Instruction decode and operand reading can be done in parallel because RISC instructions are *fixed-field* encoded.

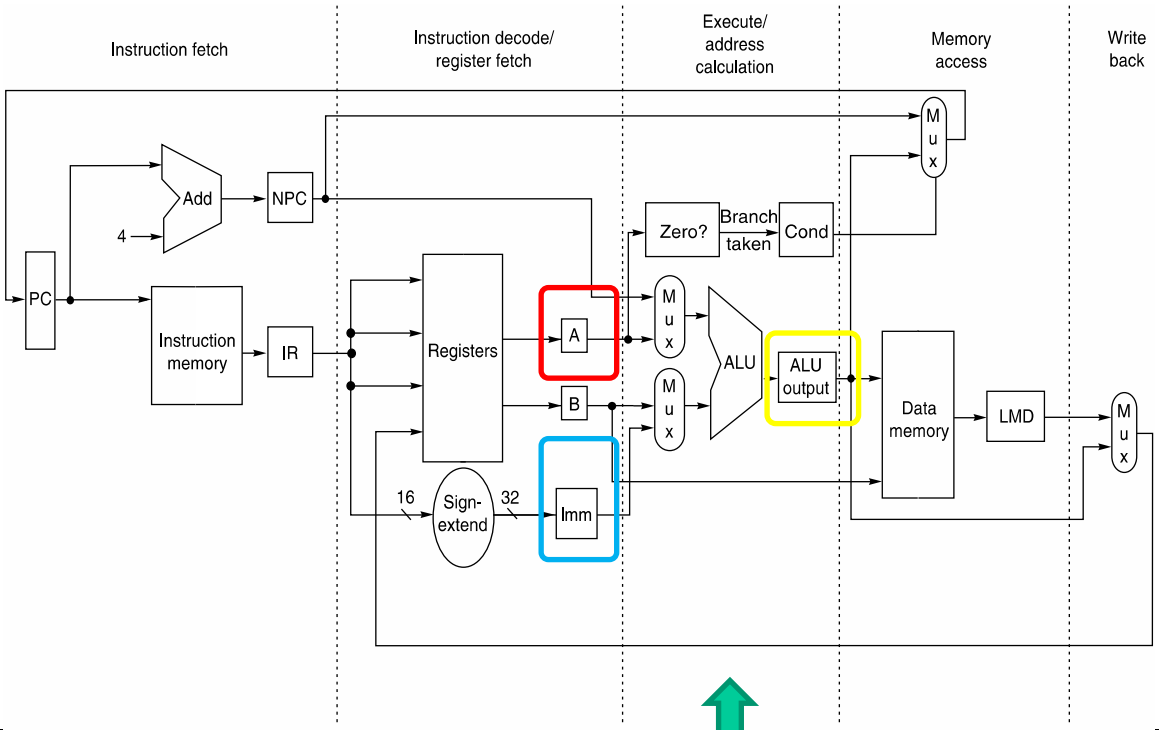
- In case of register operands read the source registers values from the register file
- In case of immediate operands, sign-extend the offset field of the instruction



Note: "useless" operations may be performed (e.g., register read if the instruction is branch; immediate value extension if no immediate). However, no added cost in terms of *time* and of *existing units*. It could be more critical in terms of power consumption.

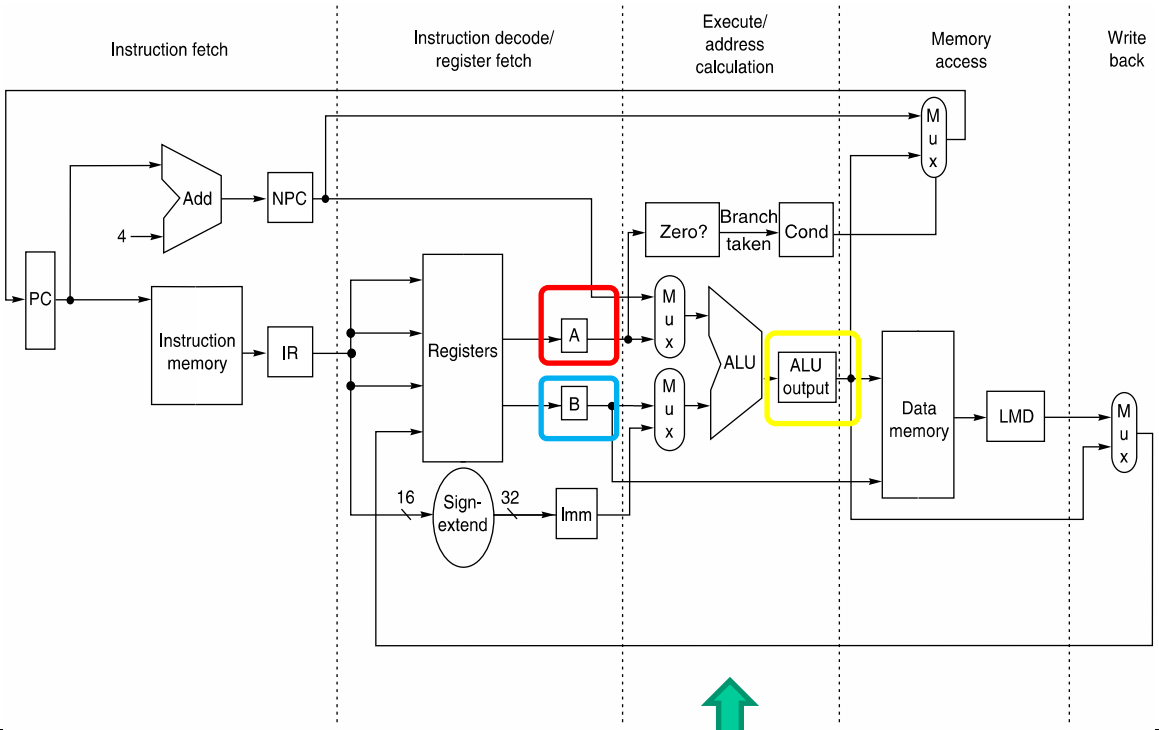
Instruction execution stages

- ***Execution/effective address stage* (EX):** The ALU operates on the operands prepared in the previous cycle, performing different operations depending on the instruction type.
  - ***Load/store instructions***: The ALU adds the base register and the offset to form the effective address



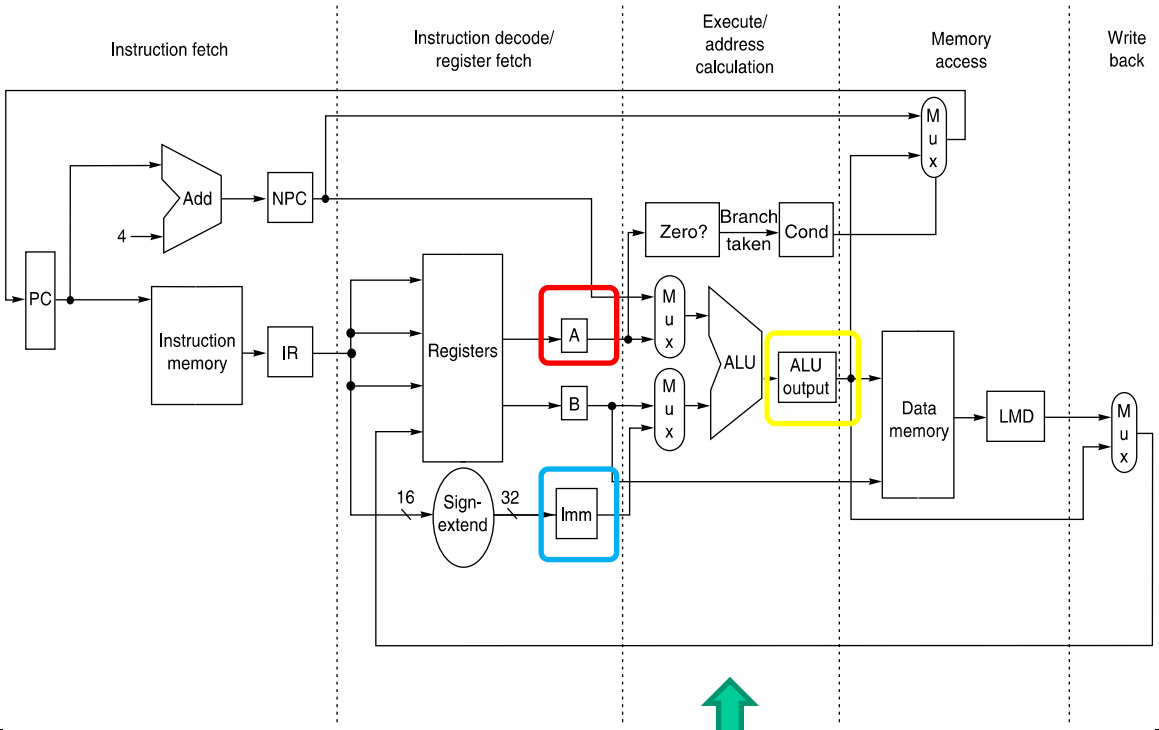
Instruction execution stages

- ***Execution/effective address stage* (EX):** The ALU operates on the operands prepared in the previous cycle, performing different operations depending on the instruction type.
  - ***Register-Register ALU instructions***: The ALU performs the operation specified in the ALU opcode on the values read from the register file



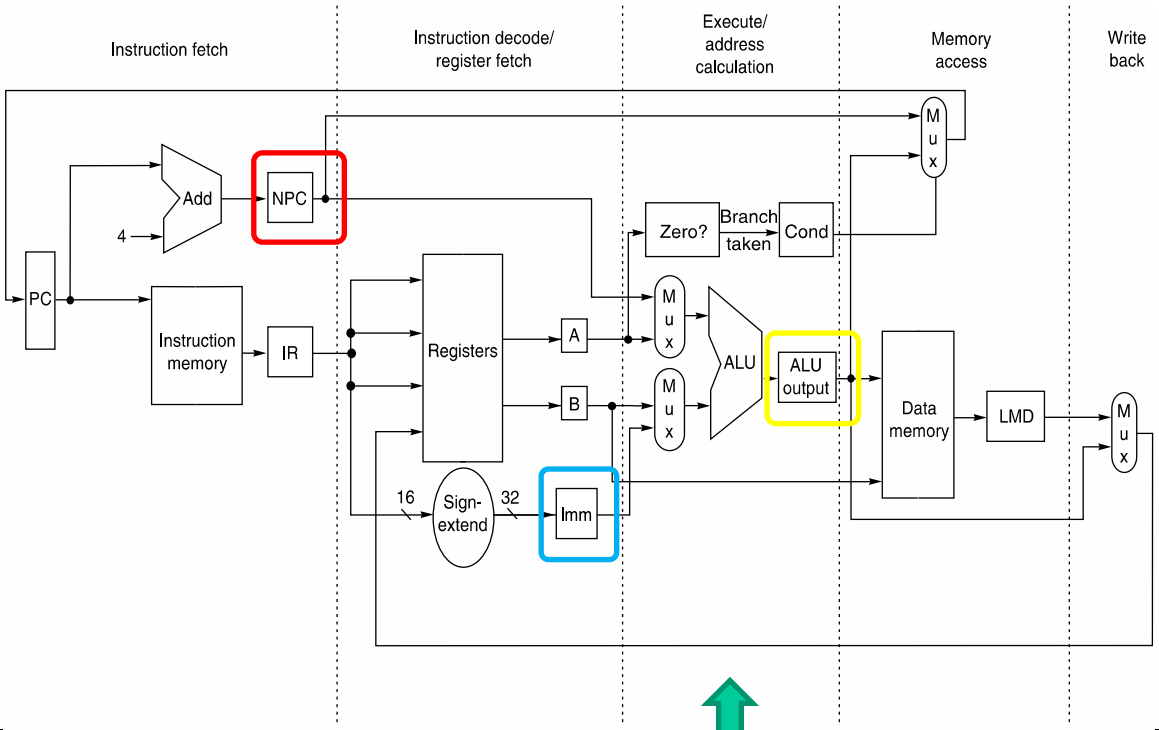
Instruction execution stages

- ***Execution/effective address stage* (EX):** The ALU operates on the operands prepared in the previous cycle, performing different operations depending on the instruction type.
  - ***Register-Immediate ALU instructions***: The ALU performs the operation specified by the ALU opcode on the value read from the specified register and the sign-extended immediate



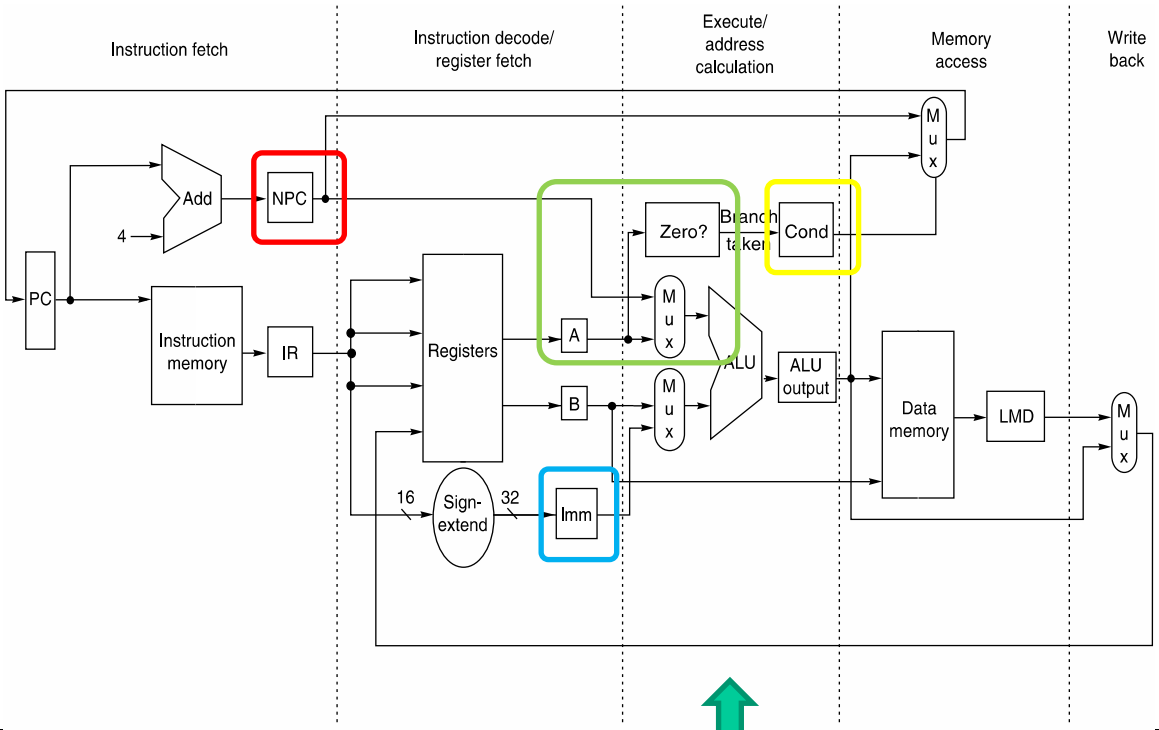
Instruction execution stages

- ***Execution/effective address stage* (EX):** The ALU operates on the operands prepared in the previous cycle, performing different operations depending on the instruction type.
  - ***Branch instructions***: The ALU adds the next instruction value to the extended value of the address (multiplied by 4) to calculate the branch target address (BTA)



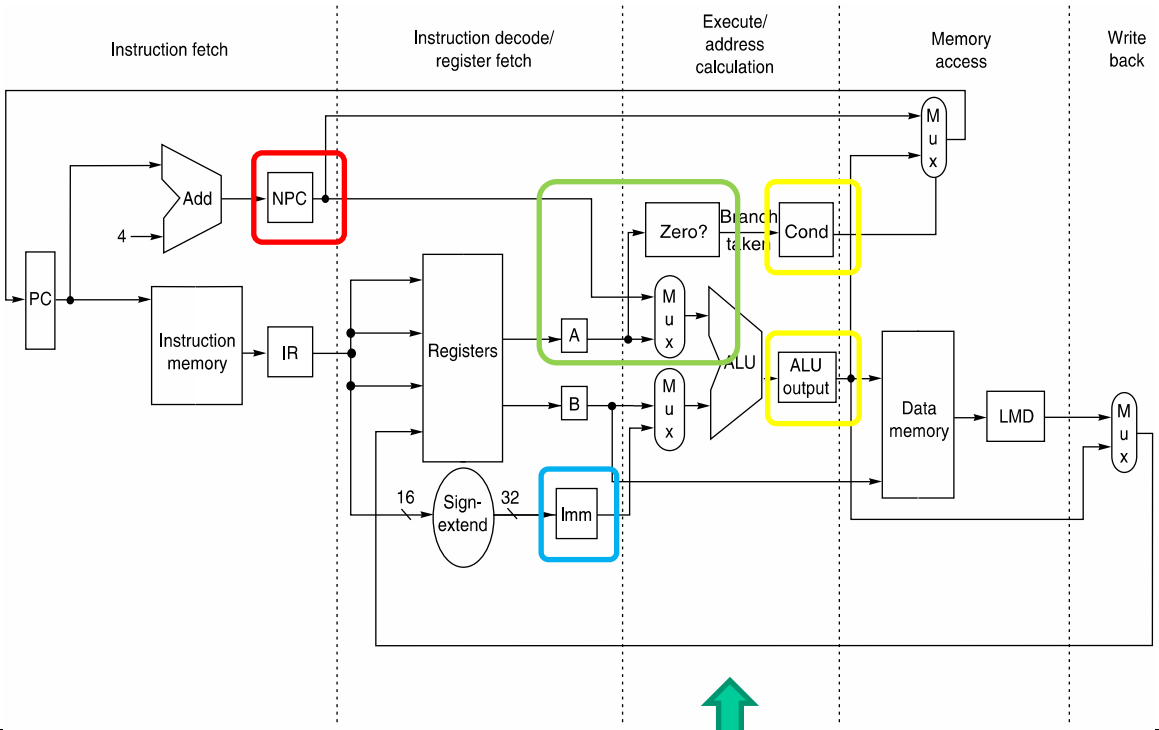
Instruction execution stages

- ***Execution/effective address stage* (EX):** The ALU operates on the operands prepared in the previous cycle, performing different operations depending on the instruction type.
  - ***Branch instructions***: In conditional branches the value of the condition register is compared with the reference value (i.e., 0) and stored into a special register



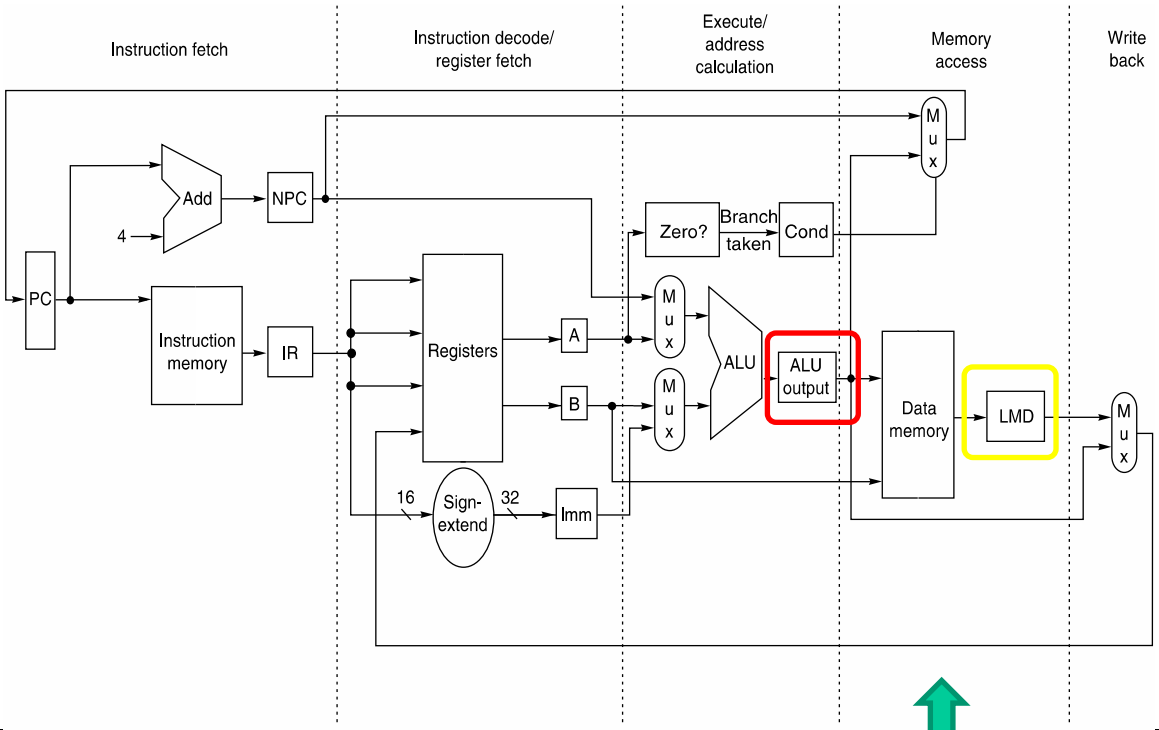
Instruction execution stages

Note: In a load-store architecture the effective address and execution cycle can be combined into a single stage because no instruction needs to simultaneously calculate a data memory address (load/store), perform an operation on the data (register-register/register-immediate), and/or calculate a Branch Target Address (branches)



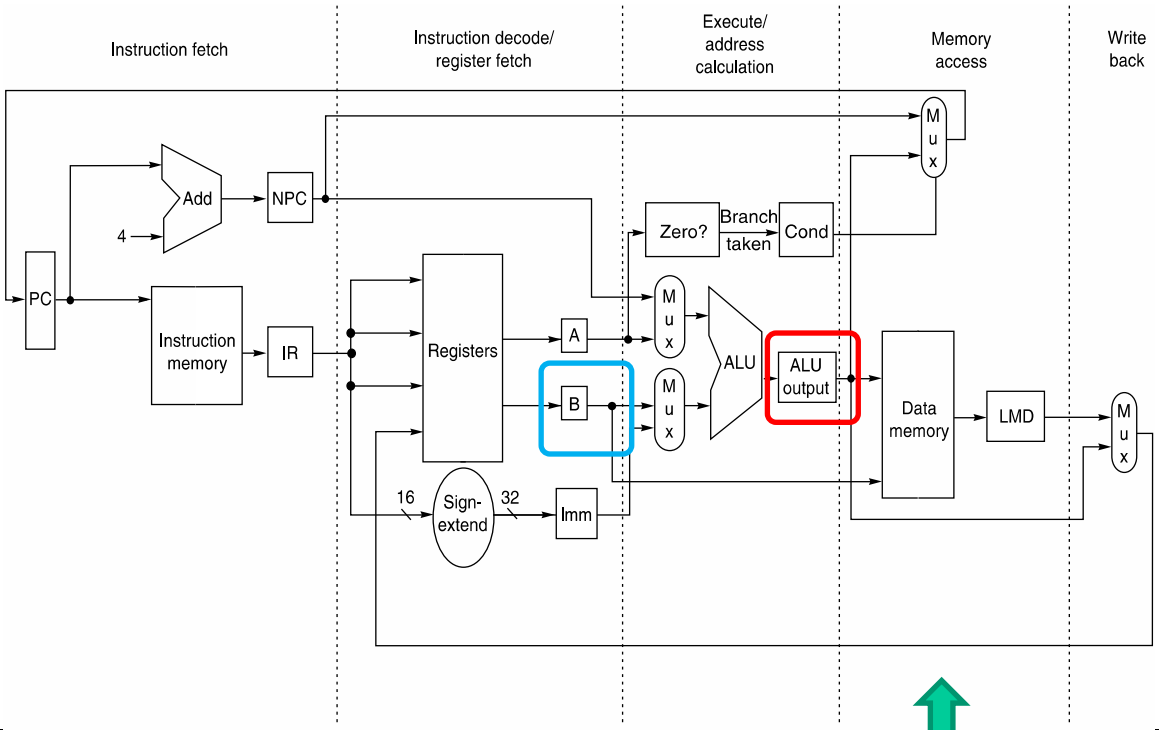
Instruction execution stages

- ***Memory access stage*** **(MEM):**
  - ***Load instructions***: The memory performs a read using the effective address computed in the previous cycle using A and Imm and saved in ALU output



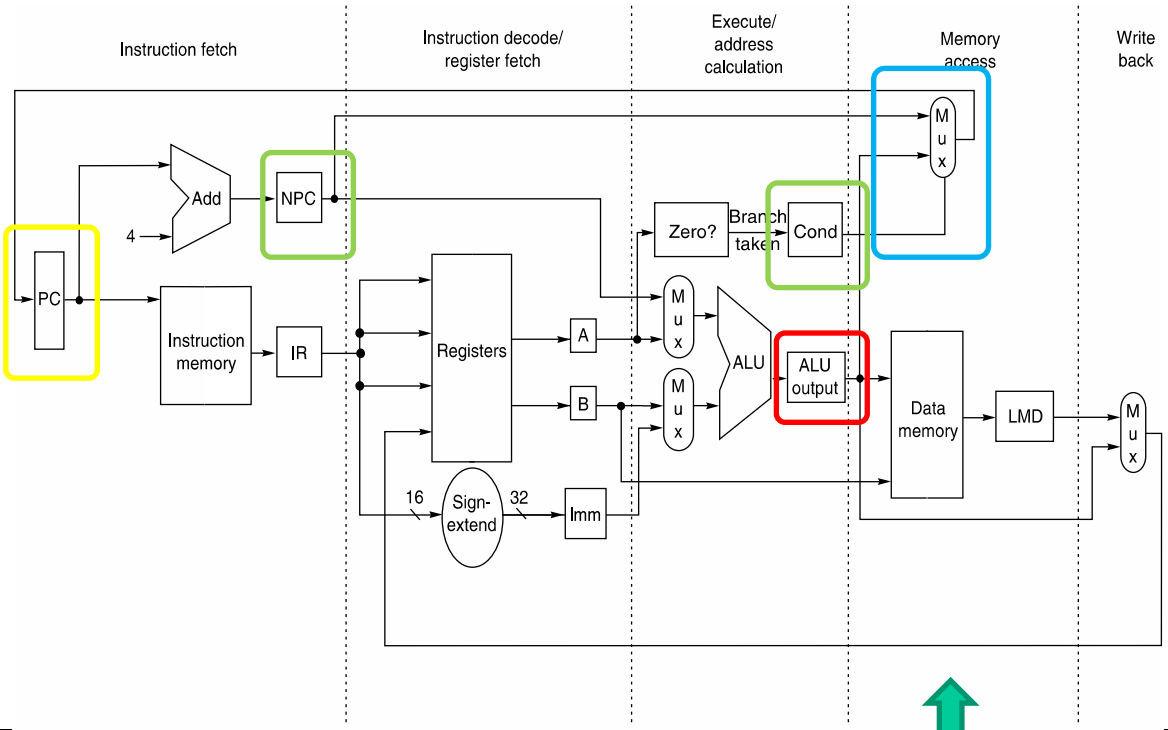
Instruction execution stages

- ***Memory access stage*** **(MEM)**:
  - ***Store instructions***: The memory performs a write using the effective address computed in the previous cycle using A and Imm and saved in ALU output, while the value comes from the other register, saved in B



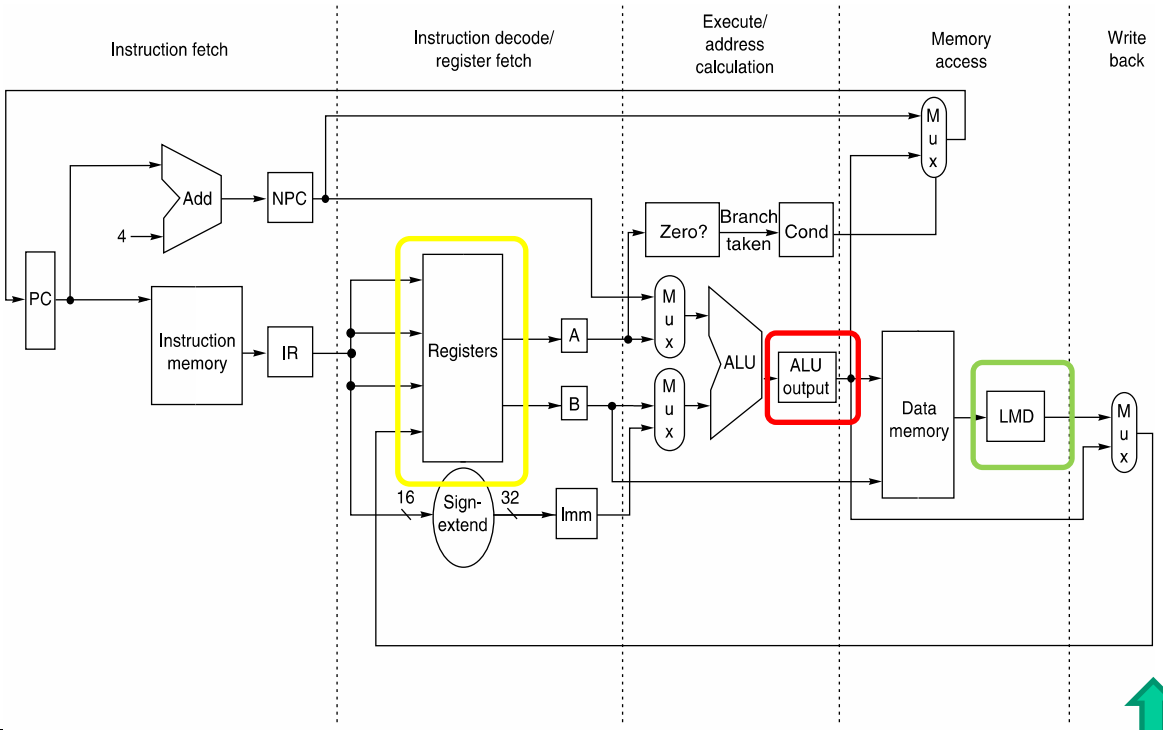
Instruction execution stages

- ***Memory access stage*** **(MEM):**
  - ***Branch instructions***: in branches, the PC is replaced with the BTA (saved in ALU output) according to the outcome of the comparison with zero, saved in Cond



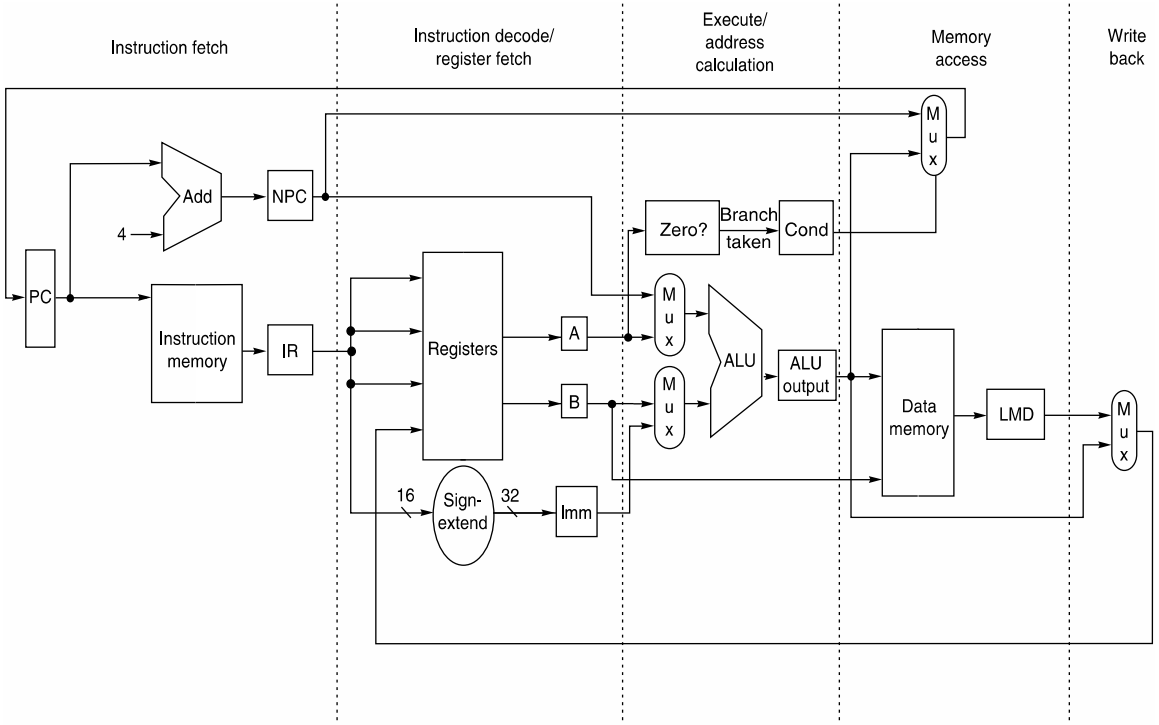
Instruction execution stages

- ***Write-back stage* (WB)**
  - ***Loads and ALU instructions***: Write the result into the register file, whether it comes from the memory system (for a load) or from the ALU (for an ALU instruction)



Instruction execution stages

- Note that if a non-pipelined implementation is used, different instructions have **variable latency**, depending on actions actually performed.
- It is important to **perform approximately the same amount of work in each stage**. In fact, the clock cycle of the processor depends on the latency of the stage with the worst-case delay



Instruction execution stages: summary

Stage	ALU instruction	Load or store instruction	Branch instruction
IF	IR = Mem[PC] NPC = PC + 4	IR = Mem[PC] NPC = PC + 4	IR = Mem[PC] NPC = PC + 4
ID	A = Rs1; B = Rs2; imm = sign-extend imm IR	A = Rs1; B = Rs2; imm = sign-extend imm IR	A = Rs1; B = Rs2; imm = sign-extend imm IR
EX	ALUoutput = A op B; or ALUoutput = A op imm	ALUoutput = A + imm	ALUoutput = NPC + imm * 4 Cond = (A == 0)
MEM		LMD = Mem[ALUoutput]; or Mem[ALUoutput] = B	If (Cond) then PC = ALUoutput
WB	Rd = ALU output	Rd = LMD	

- Notice that not all instructions require the use of every stage of the processor
- The advantage of this scheme is that it uses different functional units in each stage

Notes

- The proposed architecture executes each instruction in at most 5 clock cycles, under the supervision of a simple control unit (CU)
- At the end of each clock cycle, every value computed during that cycle is written into a storage device (memory, GPR, PC or temporary registers, such as LMD, Imm, A, B, IR, NPC, ALUOutput or Cond)
- The proposed architecture is not the only possible implementation of the given ISA, nor it is the most efficient
  - For instance, a single memory could be used for instruction and data (as they are used at different times) or a single ALU could be used to increase PC and to perform arithmetic ops → some structural hazards could arise (see later…)
  - Alternatively, we could implement the CPU so that every instruction takes 1 long clock cycle (longer than the maximum delay). In such case, the temporary registers should not be necessary, but duplicated functional units would be required (e.g., two compulsory memories for data and program) → very inefficient even if CPI=1!

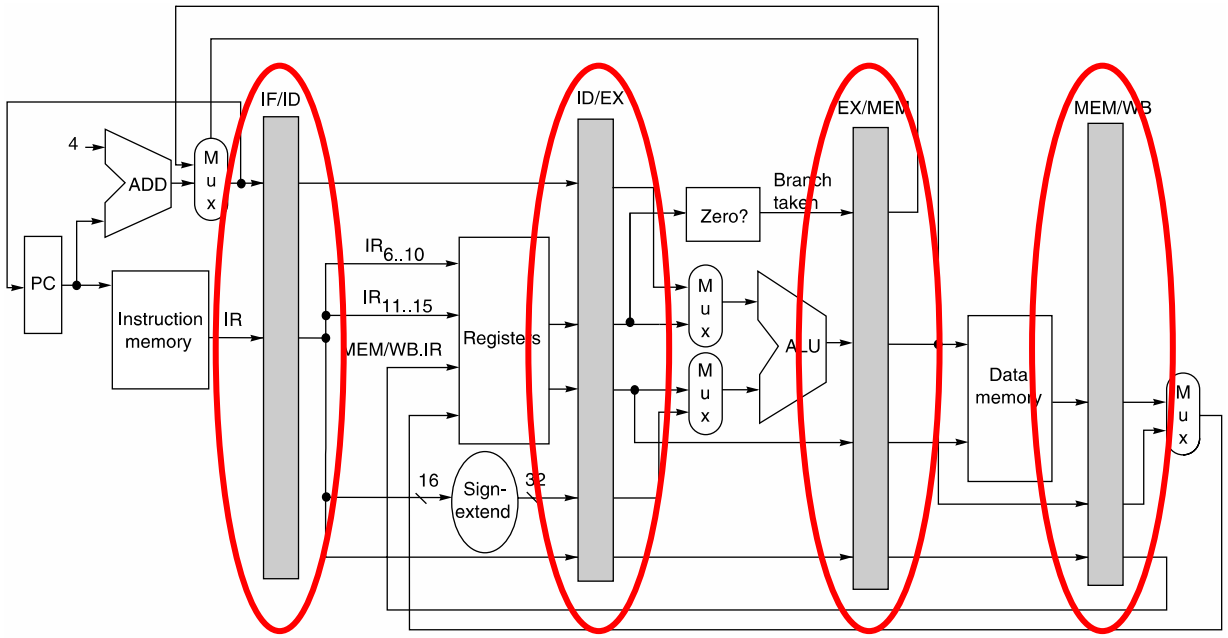
CPI computation

- Assume a 1 ns clock cycle
- Instruction latency:
  - 4 cycles for ALU operations and branches
  - 5 cycles for memory operations
- Instruction frequency:
  - ALU operations: 40%
  - Branches: 20%
  - Memory operations: 40%
- Average CPI and instruction execution time

  Average CPI = ((40% + 20%) 4 + 40% 5) = 4.4

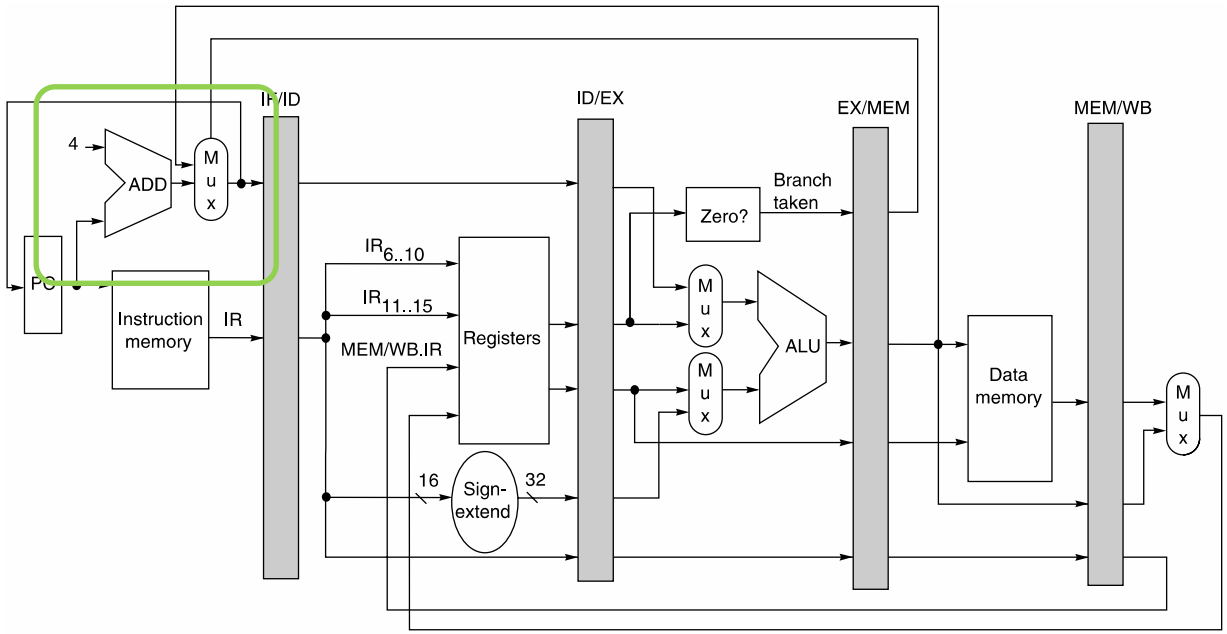
  Average Instr. Exec. Time = Clock cycle * Average CPI = 1 ns x 4.4 = 4.4 ns

Pipelining the previous RISC architecture

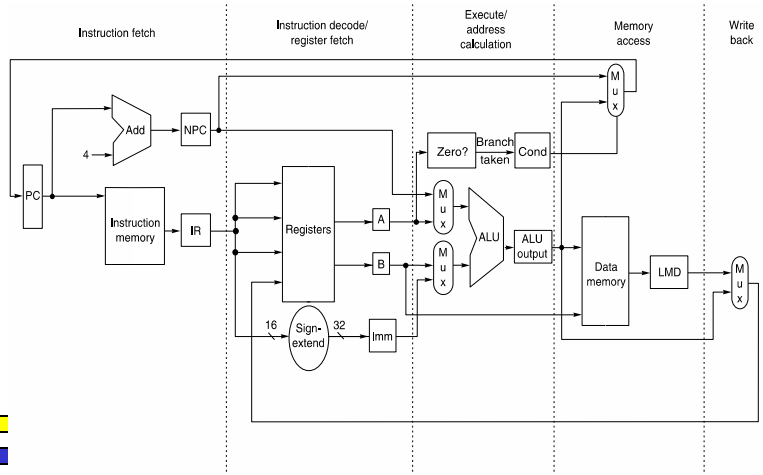


- Instructions are fetched and initiated at every clock cycle filling all the stages. In this way the CPU will be perfectly busy. For this, the following basic modifications must be applied
  - Insert homogeneous registers or latches between stages (IF/ID, ID/EX, EX/MEM, MEM/WB) thus replacing (i.e., including) temporary registers
  - Propagate the control signals generated by the CU as well as the status bits of each instruction within the same pipeline registers

Pipelining the previous RISC architecture

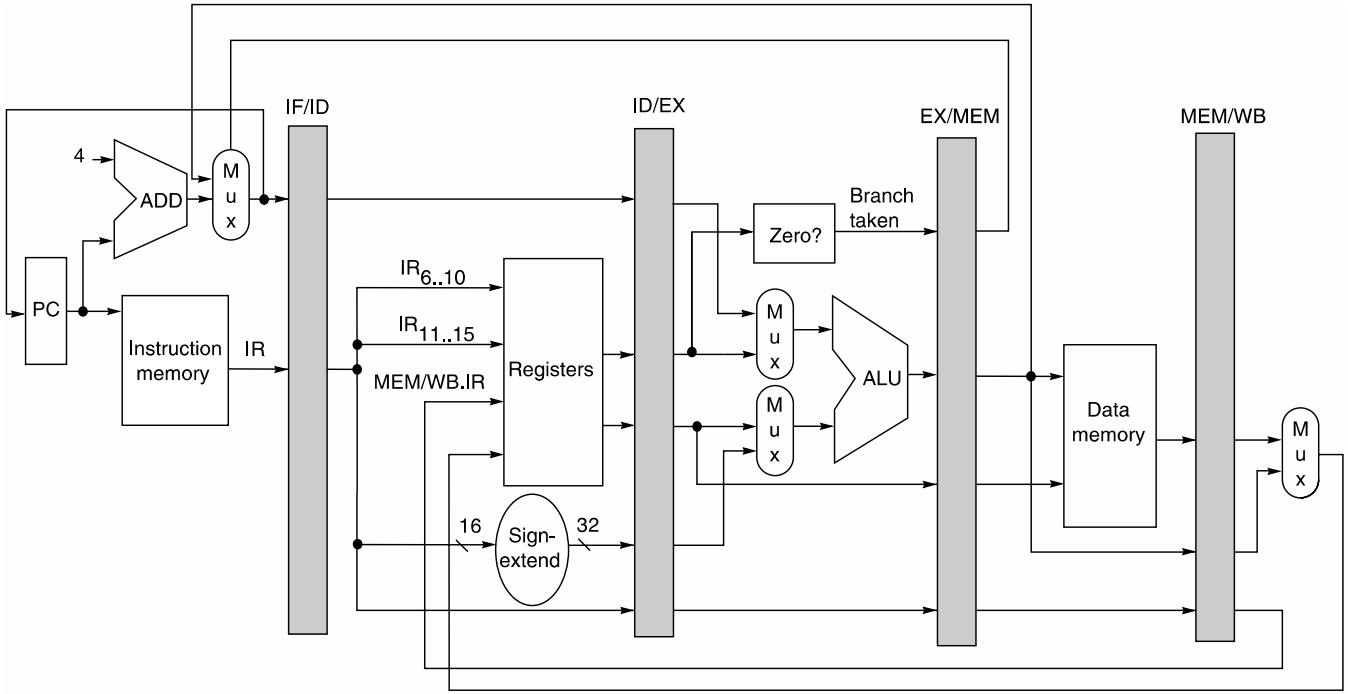


- Anticipate the PC contents update operation to the IF stage (instead of running it in the MEM stage) to avoid fetch latencies



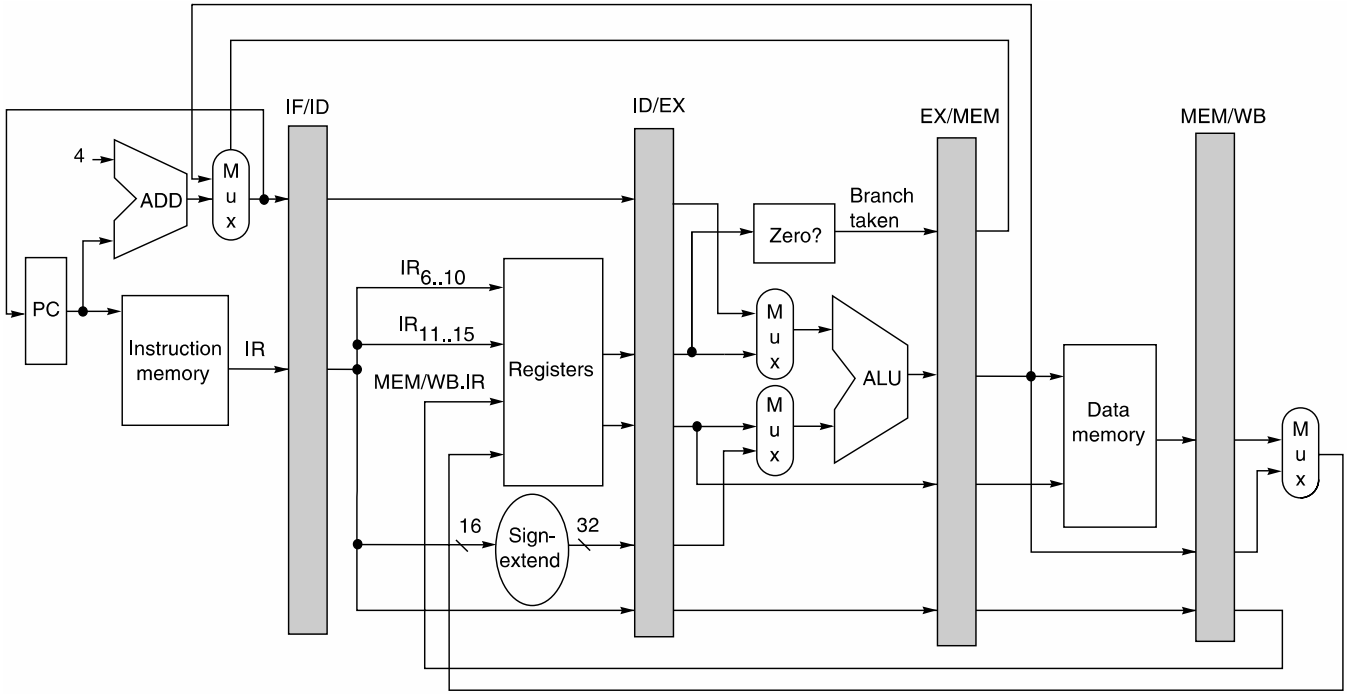
Instruction execution stages: summary

Stage	Any instruction
IF	IF/ID.IR ← Mem[PC]; IF/ID.NPC,PC ← (if ((EX/MEM.opcode == branch) & EX/MEM.cond){EX/MEM. ALUOutput} else {PC+4});
ID	ID/EX.A ← Regs[IF/ID.IR[rs]]; ID/EX.B ← Regs[IF/ID.IR[rt]]; ID/EX.NPC ← IF/ID.NPC; ID/EX.IR ← IF/ID.IR; ID/EX.Imm ← sign-extend(IF/ID.IR[immediate field]);



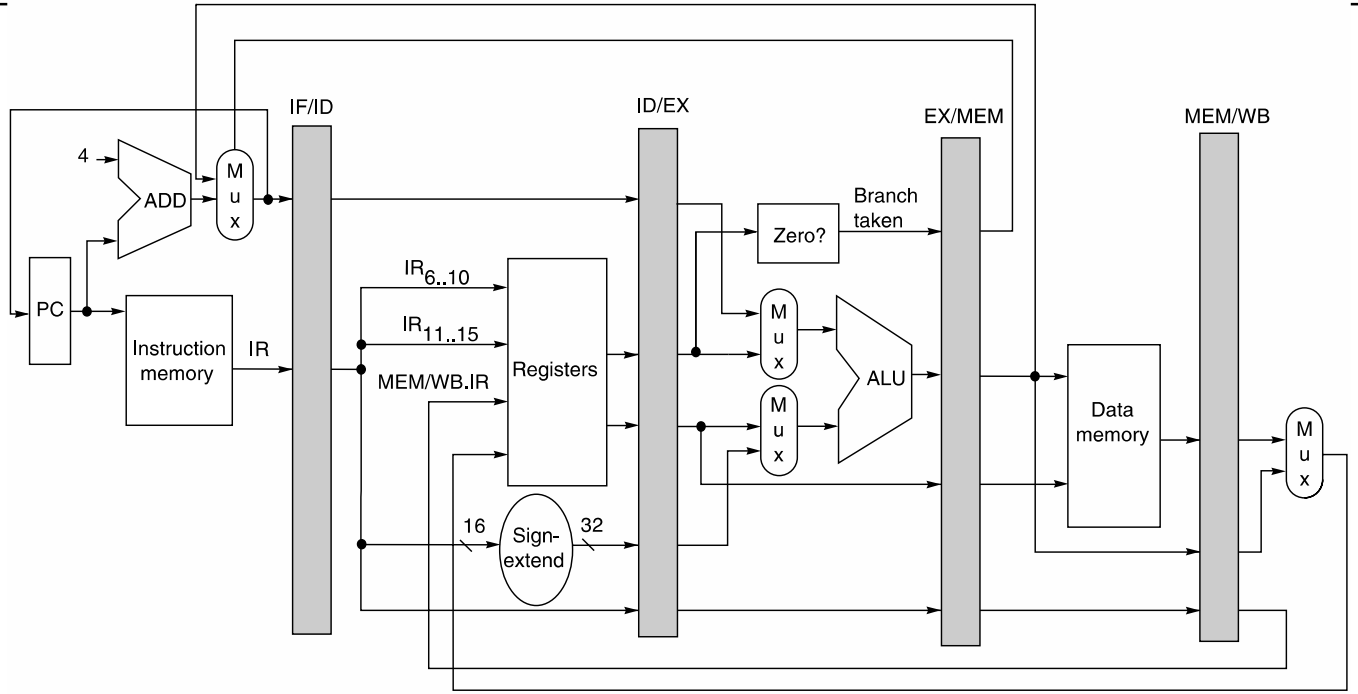
Instruction execution stages: summary

	ALU instruction	Load or store instruction	Branch instruction
EX	EX/MEM.IR ← ID/EX.IR; EX/MEM.ALUOutput ← ID/EX.A func ID/EX.B; or EX/MEM.ALUOutput ← ID/EX.A *op* ID/EX.Imm;	EX/MEM.IR to ID/EX.IR EX/MEM.ALUOutput ← ID/EX.A + ID/EX.Imm;	EX/MEM.ALUOutput ← ID/EX.NPC + (ID/EX.Imm << 2);
		EX/MEM.B ← ID/EX.B;	EX/MEM.cond ← (ID/EX.A == 0);



Instruction execution stages: summary

	ALU instruction	Load or store instruction	Branch instruction
MEM	MEM/WB.IR ← EX/MEM.IR; MEM/WB.ALUOutput ← EX/MEM.ALUOutput;	MEM/WB.IR ← EX/MEM.IR; MEM/WB.LMD ← Mem[EX/MEM.ALUOutput]; or Mem[EX/MEM.ALUOutput] ← EX/MEM.B;	
WB	Regs[MEM/WB.IR[rd]] ← MEM/WB.ALUOutput; or Regs[MEM/WB.IR[rt]] ← MEM/WB.ALUOutput;	For load only: Regs[MEM/WB.IR[rt]] ← MEM/WB.LMD;	



Instruction execution stages: summary

Stage	Any instruction		
IF	IF/ID.IR ← Mem[PC]; IF/ID.NPC,PC ← (if ((EX/MEM.opcode == branch) & EX/MEM.cond){EX/MEM.ALUOutput} else {PC+4});		
ID	ID/EX.A ← Regs[IF/ID.IR[rs]]; ID/EX.B ← Regs[IF/ID.IR[rt]]; ID/EX.NPC ← IF/ID.NPC; ID/EX.IR ← IF/ID.IR; ID/EX.Imm ← sign-extend(IF/ID.IR[immediate field]);		
	ALU instruction	**Load or store instruction**	**Branch instruction**
EX	EX/MEM.IR ← ID/EX.IR; EX/MEM.ALUOutput ← ID/EX.A func ID/EX.B; or EX/MEM.ALUOutput ← ID/EX.A *op* ID/EX.Imm;	EX/MEM.IR to ID/EX.IR EX/MEM.ALUOutput ← ID/EX.A + ID/EX.Imm; EX/MEM.B ← ID/EX.B;	EX/MEM.ALUOutput ← ID/EX.NPC + (ID/EX.Imm << 2); EX/MEM.cond ← (ID/EX.A == 0);
MEM	MEM/WB.IR ← EX/MEM.IR; MEM/WB.ALUOutput ← EX/MEM.ALUOutput;	MEM/WB.IR ← EX/MEM.IR; MEM/WB.LMD ← Mem[EX/MEM.ALUOutput]; or Mem[EX/MEM.ALUOutput] ← EX/MEM.B;	
WB	Regs[MEM/WB.IR[rd]] ← MEM/WB.ALUOutput; or Regs[MEM/WB.IR[rt]] ← MEM/WB.ALUOutput;	For load only: Regs[MEM/WB.IR[rt]] ← MEM/WB.LMD;	

Clock cycle diagram of the pipelined RISC

Instruction Number	Clock Number								
	1	2	3	4	5	6	7	8	9
i	IF	ID	EX	MEM	WB				
i+1		IF	ID	EX	MEM	WB			
i+2			IF	ID	EX	MEM	WB		
i+3				IF	ID	EX	MEM	WB	
i+4					IF	ID	EX	MEM	WB

Fill

Stable
(5 times throughput)

Drain

CPI computation

- Assume pipelining adds some overhead
  - Clock cycle increases to 1.2 ns
- Average CPI
  - Assuming ideal pipeline
  - CPI = 1
- Average instruction execution time
  - Clock cycle * Average CPI = 1.2 ns x 1 = 1.2 ns
- Speedup relative to unpipelined implementation
  - 4.4 ns / 1.2 ns = 3.7 times

Why isn't the speed-up = 5?

- The unpipelined version skips certain stages when not needed
- The pipelined version has a 20% clock overhead

Hazards and stalls

- The presence of several instructions simultaneously active into the pipeline may lead to various type of dependences which turn into ***hazards***
- *Hazards* reduce the performance from the ideal speedup by forcing the pipeline to ***stall***, i.e., to **delay subsequent instructions** until the source of the hazard is solved.
- Hazards can be solved using hardware techniques, software techniques or a combination of both

3 types of hazards

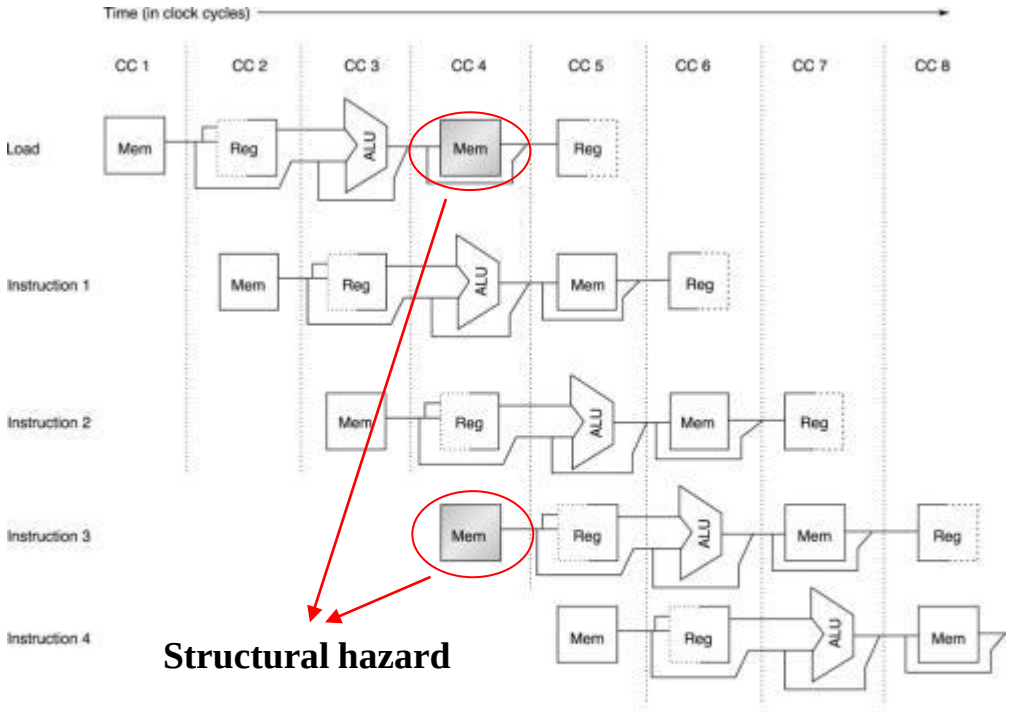
1. ***Structural hazards***: arise from resource conflicts when the hardware can not support all possible combinations of instructions at the same time
2. ***Data hazards***: arise when an instruction depends on the results of previous instructions within the pipeline which have not finished yet
3. ***Control hazards***: arise from the pipelining of branches or other instructions that change the PC content

Structural hazards

If some combination of instruction requires the use of the same resource, a *structural hazard* arises

Example

- Assume we have a single-port unique memory for both instructions and data.
- In this case a load or store instruction (*i* instruction) in the MEM stage prevents the fetch of the i+3 instruction.
- This can be solved only by halting (i.e., stalling) the pipeline for 1 instruction cycle



© 2003 Elsevier Science (USA). All rights reserved.

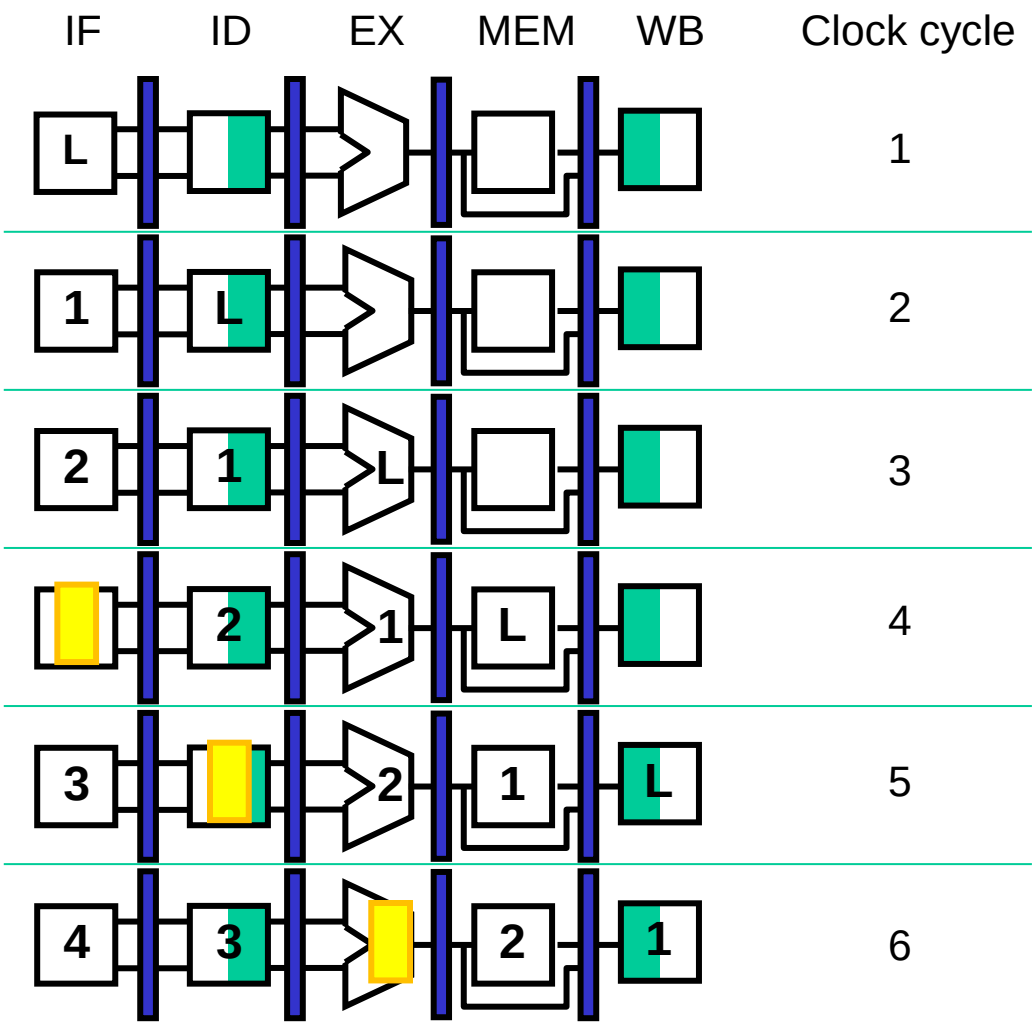
Stalling the pipeline

To avoid structural hazards (which would lead to incorrect results), the pipeline is stalled → 1 or more "empty" cycles or *bubbles* are inserted in the pipeline

	Clock cycle number									
Instruction	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**
Load instruction	IF	ID	EX	MEM	WB					
Instruction $i + 1$		IF	ID	EX	MEM	WB				
Instruction $i + 2$			IF	ID	EX	MEM	WB			
Instruction $i + 3$				stall	IF	ID	EX	MEM	WB	
Instruction $i + 4$						IF	ID	EX	MEM	WB
Instruction $i + 5$							IF	ID	EX	MEM
Instruction $i + 6$								IF	ID	EX

Note: if $i + 1$ and $i + 2$ are also loads, we should really be stalling for 3 clock cycles!

Stalling the pipeline



Instruction	Clock cycle number									
	1	2	3	4	5	6	7	8	9	10
Load instruction	IF	ID	EX	MEM	WB					
Instruction *i* + 1		IF	ID	EX	MEM	WB				
Instruction *i* + 2			IF	ID	EX	MEM	WB			
Instruction *i* + 3				stall	IF	ID	EX	MEM	WB	
Instruction *i* + 4						IF	ID	EX	MEM	WB
Instruction *i* + 5							IF	ID	EX	MEM
Instruction *i* + 6								IF	ID	EX

Solving structural hazards

To avoid structural hazards (which would lead to incorrect results), the pipeline is stalled → 1 or more "empty" cycles or *bubbles* are inserted in the pipeline

Stalls result in a reduction of instruction throughput.

Possible solutions

- *Hardware*: Resource splitting/replication (e.g., in the previous example a data cache provided with separate *Read* and *Write* buffers could be used) → expensive. If the conflict is rare it is not worth replicating a certain unit
- *Software*: Instructions requesting the same resource are located at proper distance from each other. For instance, two conflicting instructions could be delayed by the compiler → effective, but compiler design could be cumbersome

Impact of hazards on CPI

Say the probability of a stall is 20%: 1 stall every 5 instructions

$$CPI = \frac{\#clocks}{\#instr} = \frac{IC + \left(\frac{IC}{5} - 1\right)}{IC} = 1 + \frac{1}{5} - \frac{1}{IC}$$

Asymptotically, the last term can be neglected

$$CPI = 1 + \frac{1}{5} = 1 + 0.2$$

The CPI is increased by the average number of stalls per instruction (same as the probability of a stall)

Data dependences and hazards

- Data hazards are related to *data dependences*, i.e., sequential constraints imposed by the data flow of the program
- Two different instructions are *data dependent* if they have a common register or memory location; unless the operand is for both a *source* operand
- Data dependences can occur in:
  - ***Straight-line code***: between subsequent instructions during the execution of a program segment
  - ***Loops:*** between instructions belonging to different iterations of the loop (also called ***recurrences*** or ***inter-iteration dependences***)
    - *Example: first-order linear recurrences:* e.g.: X(i) = A(i) * X(i-1) + B(i)
- Three kinds of data dependences:
  - **Read after Write (RAW) also called flow or true dependence**
  - **Write after Read (WAR) also called anti dependence**
  - **Write after Write (WAW) also called output dependence**

Types of data dependences and hazards

- **RAW dependence:** a ***read*** uses the value written by a ***write*** instruction.

```
I: add r1,r2,r3
J: sub r4,r1,r3
```

  – **Hazard:** Instruction J tries to read an operand **before** instruction I has written it.

- **WAR hazard**: the destination of an instruction is a source operand of a previous instruction

```
I: sub r4,r1,r3
J: add r1,r2,r3
```

  – **Hazard:** Instruction J writes operand **before** instruction I reads it. This results from reuse of the name "r1".

- **WAW hazard**: two instructions have the same destination operand.

```
I: sub r1,r4,r3
J: add r1,r2,r3
```

  – **Hazard:** Instruction J writes operand **before** Instr I writes it. This also results from the reuse of name "r1".

Hints on data hazard solutions

- WAR, WAW are ***false dependences*** (i.e., due to re-use of registers and/or memory addresses by compiler). They can be solved through ***renaming*** (during compilation or by means of specialized hardware). They do not exist in our simple 5-stage pipelined processor

Example

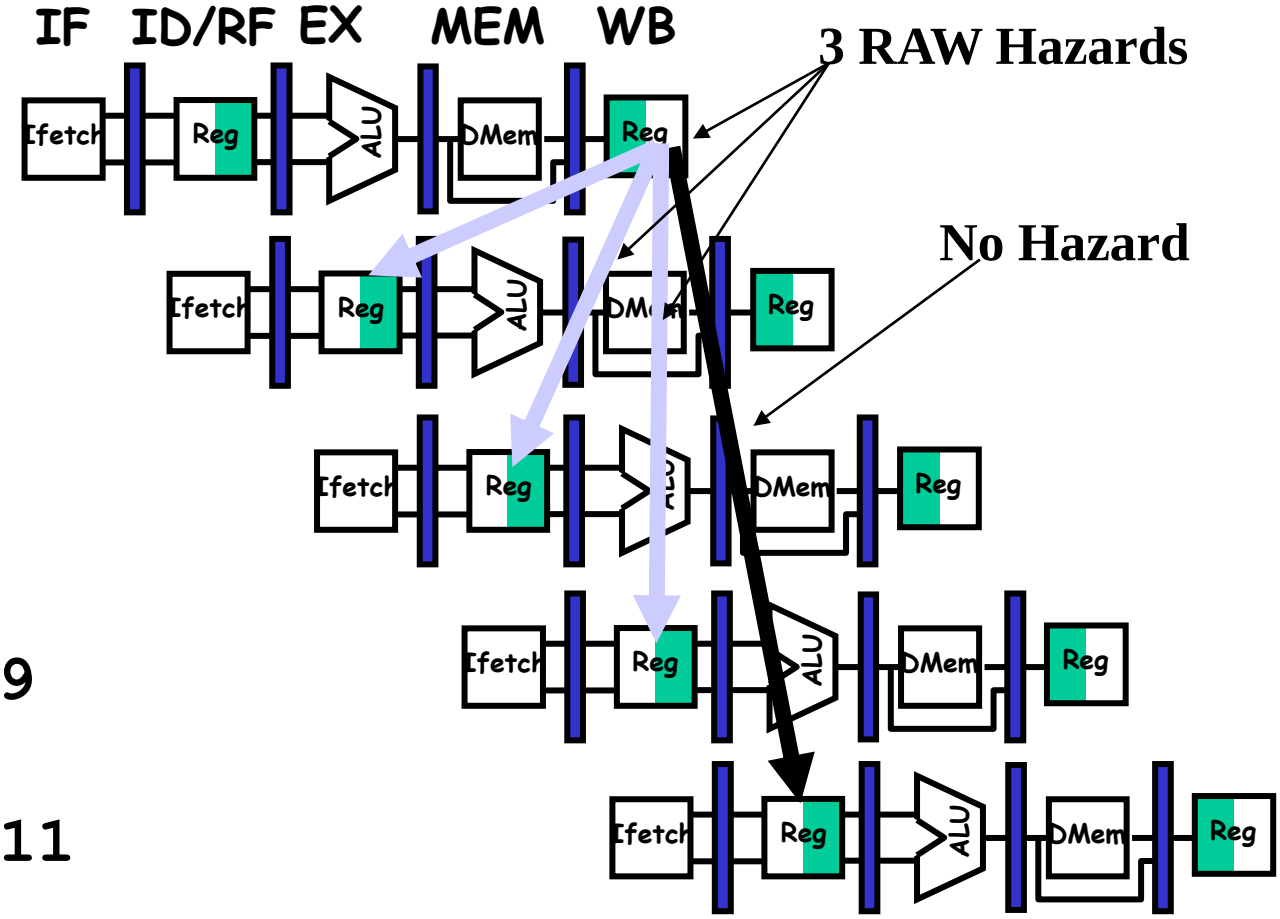
i_k	*mul r4, r1, r2*	→	i_k	*mul r4, r1, r2*
i_{k+l}	*add r1, r5, r3*	→	i_{k+l}	*add r6, r5, r3*

- RAWs are ***true dependences*** (i.e., related to the semantics of the program). RAW hazards cannot be solved but correctness can be assured at software level or through hardware techniques (e.g., *forwarding* see later)

RAW hazard example in a 5-stage pipeline

Instr. Order

```
add r1,r2,r3
sub r4,r1,r5
and r6,r1,r7
or   r8,r1,r9
xor r10,r1,r11
```



Software solutions to RAW hazards

- *Code moving*: If possible, the compiler inserts as many non-dependent instructions as necessary (3 instructions in the case considered) between *define* and first *use* of a variable

```
sub   r2, r1, r3
and   r12, r2, r5
or    r13, r6, r2
add   r24, r20, r20
and   r7, r8, r9
lw    r16, 100(r18)
```

→

```
sub   r2, r1, r3
add   r24, r20, r20
and   r7, r8, r9
lw    r16, 100(r18)
and   r12, r2, r5
or    r13, r6, r2
```

- *Nop stuffing*: The compiler inserts as many *nop* as are the required cycles between define and first use → ***Nops*** do not perform any active computation → equivalent frequency of execution ***decreases***

```
sub   r2, r1, r3
and   r12, r2, r5
or    r13, r6, r2
```

→

```
sub   r2, r1, r3
nop
nop
nop
and   r12, r2, r5
or    r13, r6, r2
```

Managing RAW hazards using the CU

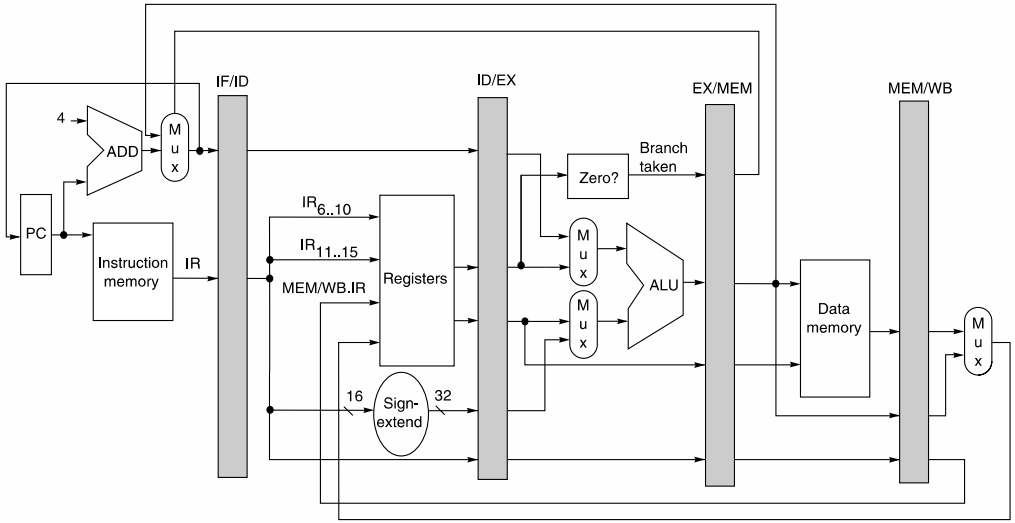
In order to manage the RAW hazard by hardware, the CU of the pipeline:

1. Check (in the ID stage) whether one of the source register r_{s1} and r_{s2} of the considered instruction I_{k+1} coincides with the destination register r_d of some previous incomplete instruction. 6 comparisons are required:

$$\text{IF/ID.IR}(r_{s1}) == \text{ID/EX.IR}(r_d) \;||\; \text{IF/ID.IR}(r_{s2}) == \text{ID/EX.IR}(r_d)$$

$$\text{IF/ID.IR}(r_{s1}) == \text{EX/MEM.IR}(r_d) \;||\; \text{IF/ID.IR}(r_{s2}) == \text{EX/MEM.IR}(r_d)$$

$$\text{IF/ID.IR}(r_{s1}) == \text{MEM/WB.IR}(r_d) \;||\; \text{IF/ID.IR}(r_{s2}) == \text{MEM/WB.IR}(r_d)$$



Managing RAW hazards using the CU

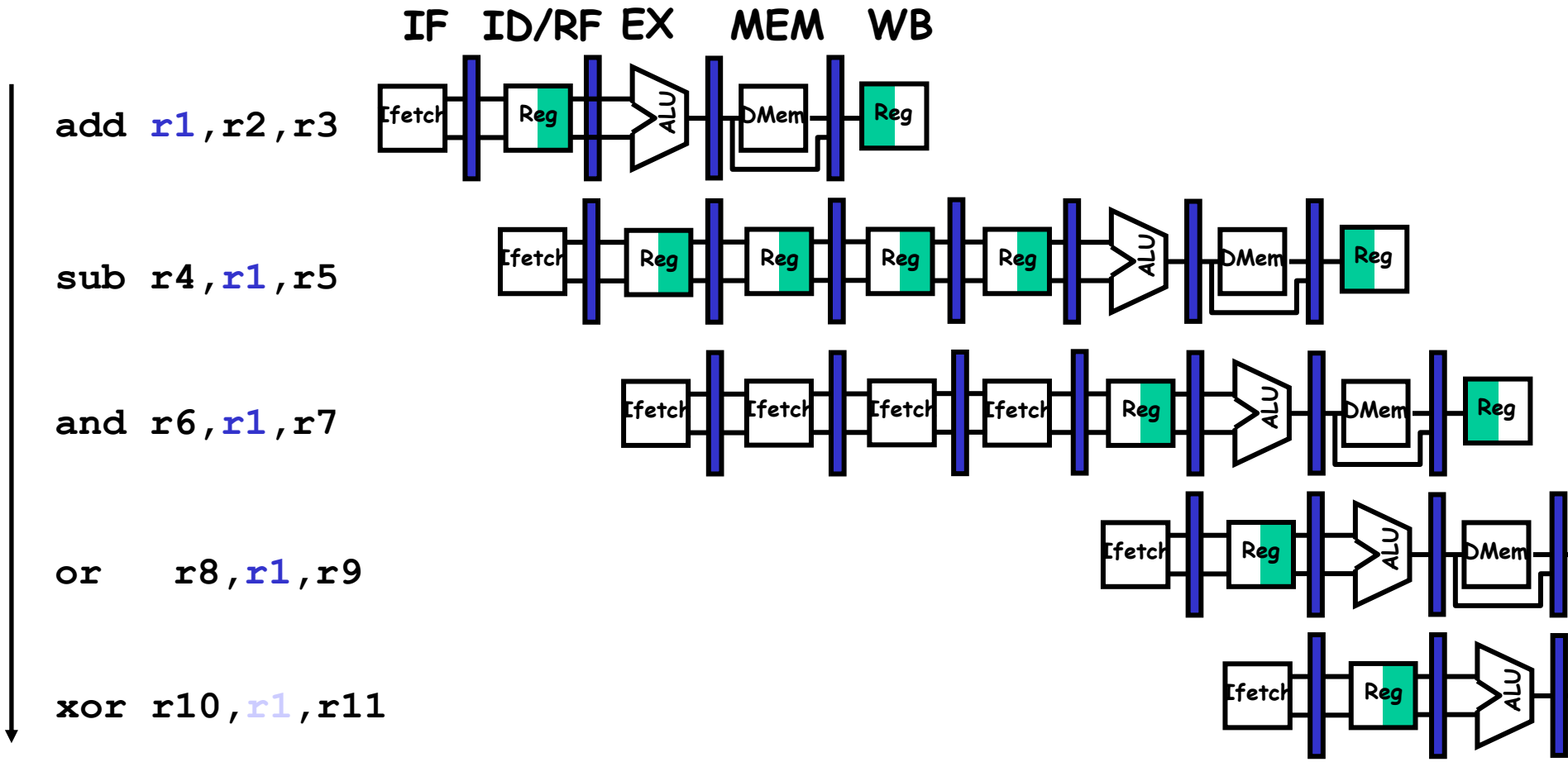
In order to manage the RAW hazard by hardware, the CU of the pipeline:

1. Check (in the ID stage) whether one of the source register r_{s1} and r_{s2} of the considered instruction I_{k+1} coincides with the destination register r_d of some previous incomplete instruction. 6 comparisons are required:

$$\text{IF/ID.IR}(r_{s1}) == \text{ID/EX.IR}(r_d) \;||\; \text{IF/ID.IR}(r_{s2}) == \text{ID/EX.IR}(r_d)$$
$$\text{IF/ID.IR}(r_{s1}) == \text{EX/MEM.IR}(r_d) \;||\; \text{IF/ID.IR}(r_{s2}) == \text{EX/MEM.IR}(r_d)$$
$$\text{IF/ID.IR}(r_{s1}) == \text{MEM/WB.IR}(r_d) \;||\; \text{IF/ID.IR}(r_{s2}) == \text{MEM/WB.IR}(r_d)$$

2. If a hazard is detected, the CU *stalls* the conflicting instruction I_{k+1} in the ID stage until the instruction I_k has completed its flow (*pipeline interlock*)
   - In practice, no compiler action is required → this is equivalent to a *nop* insertion, the only advantage being no increase of instruction memory space.
   - Control information is carried in the pipeline registers → In case of stall, the contents of IF/ID registers is re-circulated and the control portion of ID/EX, EX/MEM and MEM/WB registers are changed to all 0s → nothing is done

RAW hazard example in a 5-stage pipeline



Performance with RAW hazard

Assume there is a data dependency every 6 instructions, with 3 stalls

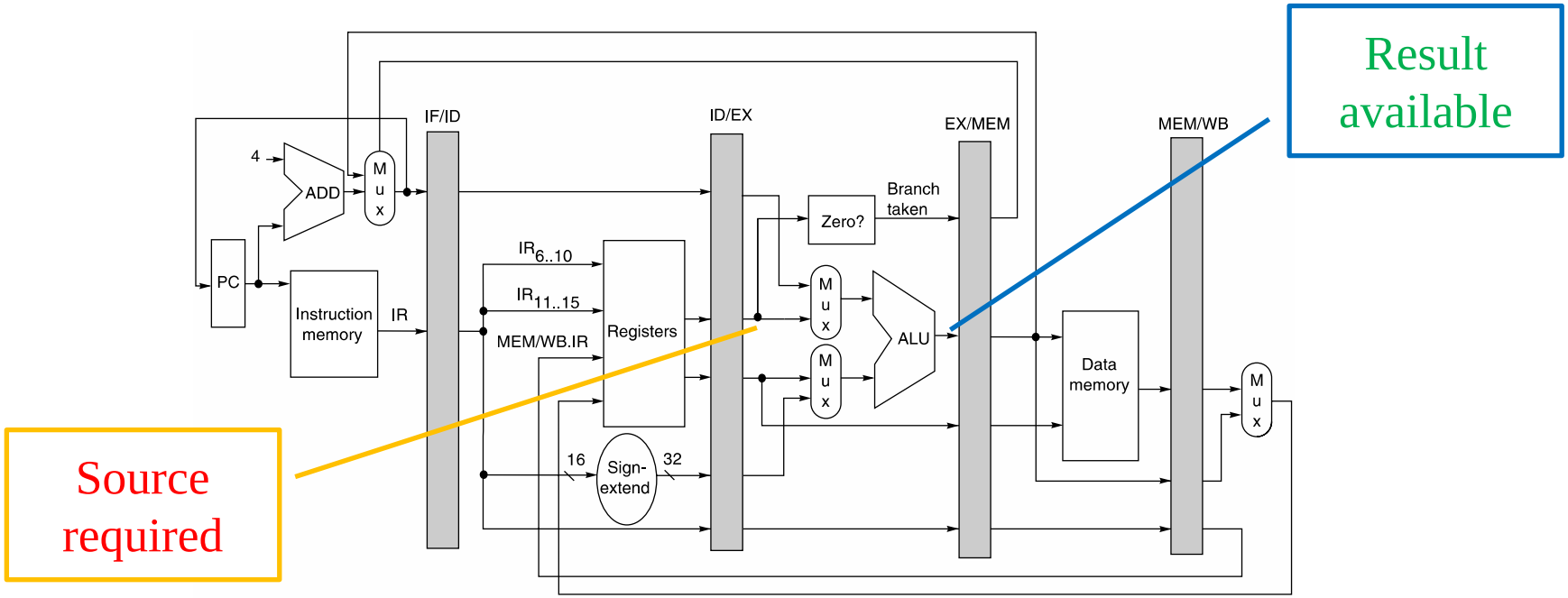
Average stalls per instruction = 3 / 6 = 0.5

CPI = 1 + 0.5 = 1.5

The CPI is 50% worse!

Stall penalty reduction through forwarding

- Note that in general:
  - ***Source operands*** are actually necessary on ALU inputs at the beginning of EX stage for instruction I_{k+1}
  - ***Destination operand*** is actually available at ALU output immediately at the end of EX stage for instruction I_k



Stall penalty reduction through forwarding

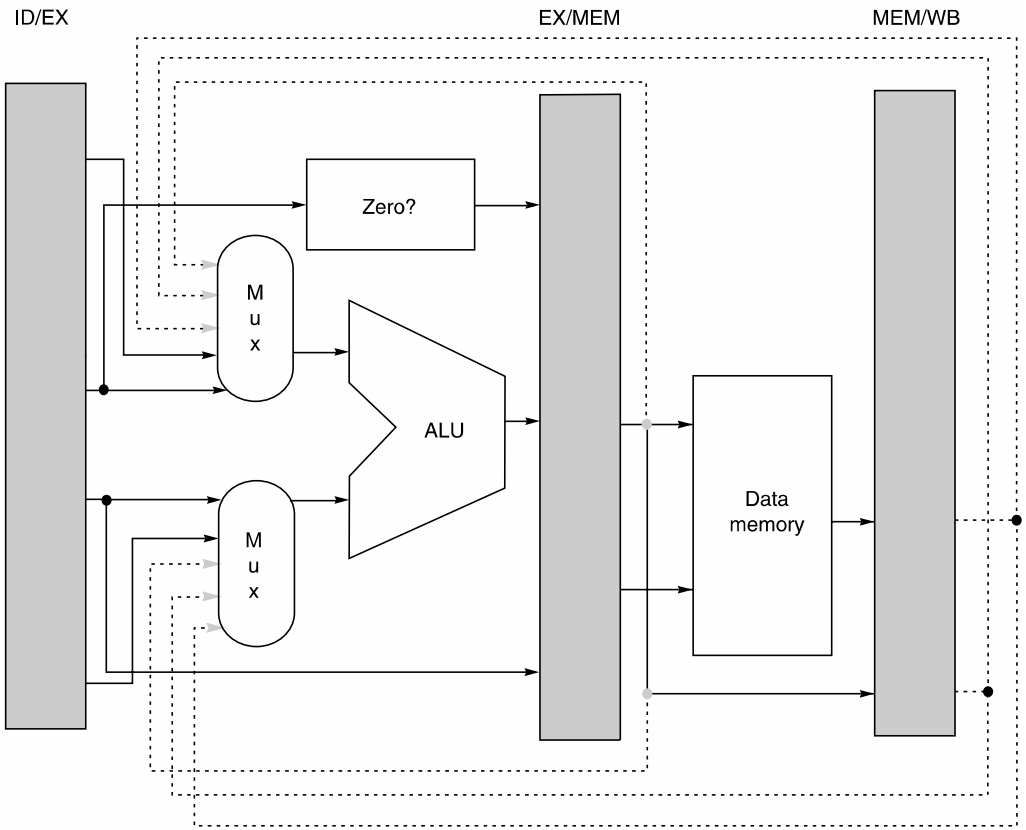
- Note that in general:
  - *Source operands* are actually necessary on ALU inputs at the beginning of EX stage
  - *Destination operand* is actually available at ALU output immediately at the end of EX stage

1. Insert bypass paths feeding the ALU output from both the EX/MEM and MEM/WB pipeline registers to the ALU inputs
2. The hardware detects that the instruction i_k currently in the EX stage requires a source register r_{s1} which should be written by the instructions i_{k-1} or i_{k-2} in the MEM and WB stages, respectively

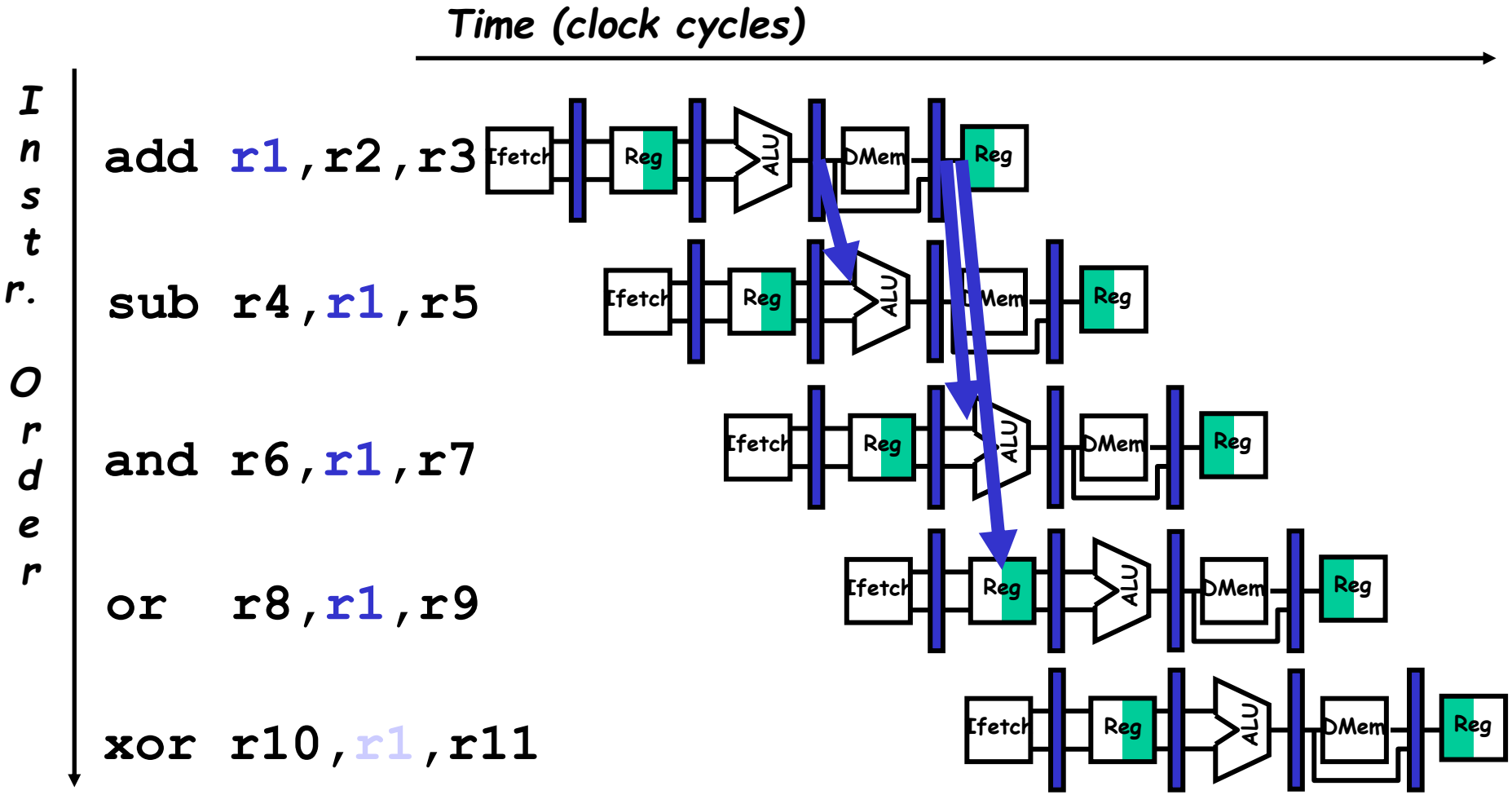
3. In this case, the control logic selects the bypassed (or forwarded) result as the ALU input rather than the value read from the register file. In practice, dependences are identified *not with respect to register file* but *with respect to inter-stage pipeline registers*.

Forwarding implementation

- This result can be generalized to include passing a result directly to the functional unit that requires it
- All forwarding logically happens from the ALU or data memory output to:
  - ALU input
  - Data memory input
  - Zero detection unit
- Thus we can implement forwarding control by multiple comparisons between EX/MEM.IR(rd) and MEM/WB.IR(rd) with ID/EX.IR(rs) and EX/MEM.IR(rs)
- Increasing HW complexity due to comparators and MUX enlargement

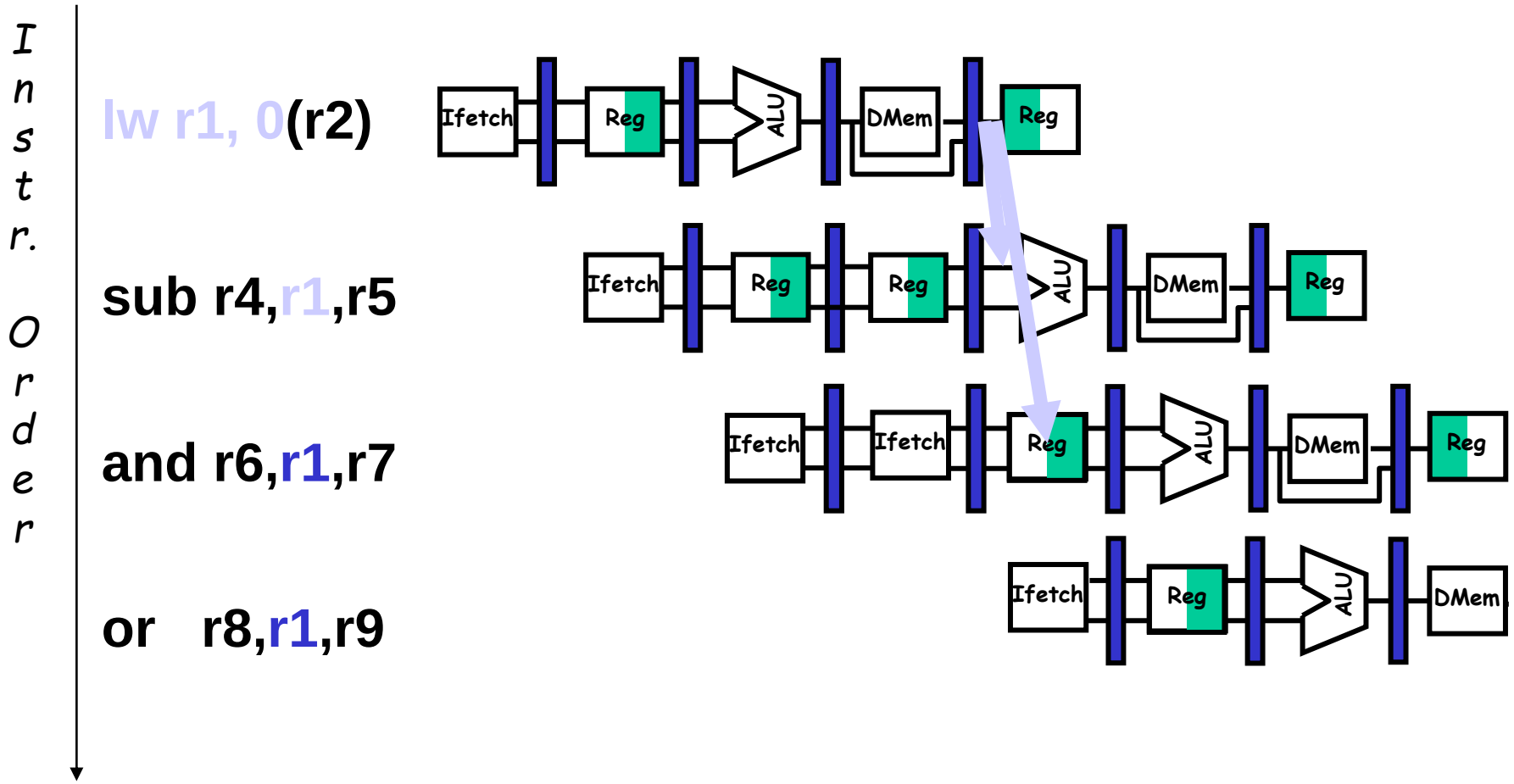


Example of forwarding - 1



Example of forwarding - 2

- Not all RAW hazard can be handled by forwarding. This happens when certain data is required at ALU inputs <u>before</u> it is ready. In this case forwarding reduces stall penalty, but it can not prevent some "bubbles"



Control dependences and hazards

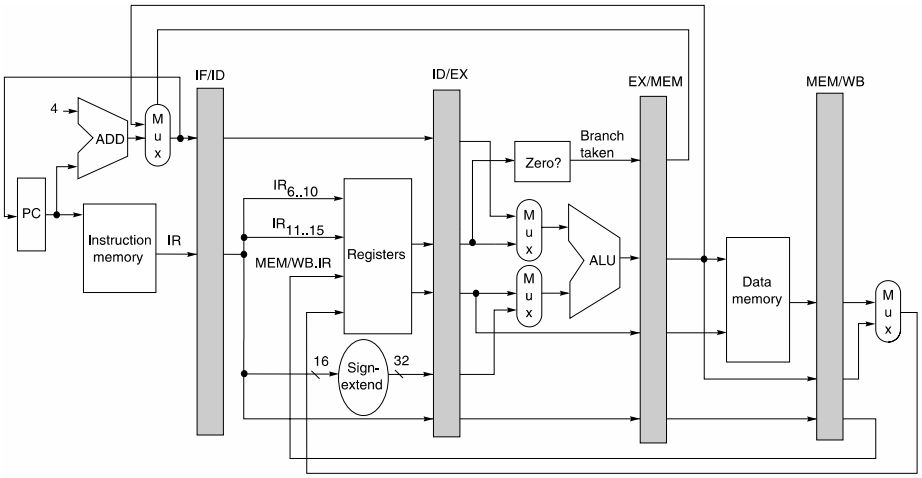
- *Control dependences* are sequential constraints due to the control flow of the program
- All branch instructions (e.g., jumps, procedure calls and returns, traps, interrupt) create control dependences
- The ratio between control dependences and straight-line code depends on the application
  - **Example:** In DSP applications control dependences are less critical than in general-purpose applications, because DSP algorithms are very regular

Control dependences and hazards

- If a branch changes the PC, it is referred to as a **taken** branch; if it falls through, it is **not taken**, or **untaken**
- *Control hazards* are related to the risk of fetching into the pipeline the wrong instructions following a conditional branch before knowing if the branch is to be taken or not
- Control hazards may seriously decrease CPU performance

Example

Given the sequence:



40	**beq**	r1, r3, #72	;cycle 1
44	**and**	r12, r2, r5	;cycle 2
48	**or**	r13, r6, r2	;cycle 3
52	**add**	r14, r2, r2	;cycle 4
.....			
72	**lw**	r4, 50(r7)	; this is the branch destination

- Actual address of "next instruction to be executed" is available in cycle 4, when *beq* enters the MEM stage
- If branch has to be taken, new address is transferred to PC at the end of cycle 4
- In the meantime, three instructions following *beq* have been fetched and forwarded through pipeline → results wrong → *control hazard*

Simple control hazard solutions - 1

- There are many methods to deal with pipeline hazards caused by branch delay. There are 4 simple static solutions

- ***Solution 1 - Freeze or flush the pipeline*** → Hold or delete any instruction after the branch until the BTA is known → very simple
- ***Solution 2 - Predict branch as not taken*** → As PC+4 is already calculated, the hardware continues to fetch instructions in sequence as if the branch were not executed. If the branch is actually taken, wrongly fetched instructions have to be "flushed" (i.e., turned into nops)

Untaken branch instruction	IF	ID	EX	MEM	WB				
Instruction i + 1		IF	ID	EX	MEM	WB			
Instruction i + 2			IF	ID	EX	MEM	WB		
Instruction i + 3				IF	ID	EX	MEM	WB	
Instruction i + 4					IF	ID	EX	MEM	WB
Taken branch instruction	IF	ID	EX	MEM	WB				
Instruction i + 1		IF	ID	EX	nop	nop			
Instruction i + 2			IF	ID	nop	nop	nop		
Instruction i + 3				IF	nop	nop	nop	nop	
Branch target					IF	ID	EX	MEM	WB
Branch target + 1						IF	ID	EX	MEM

← **Not taken**

← **Taken**

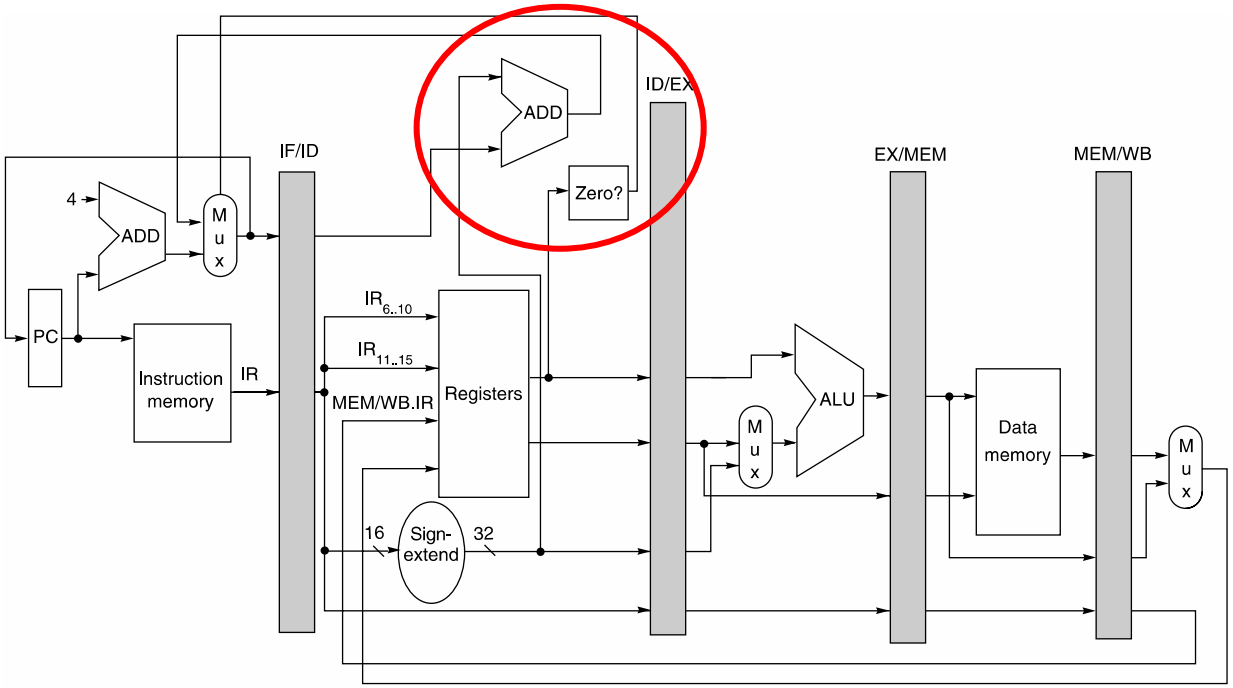
Simple control hazard solutions - 2

- ***Solution 3*** - *Predict Branch as Taken* → As soon as the branch is decoded and the BTA is calculated, we begin fetching instructions from BTA. If the branch was not to be taken, wrongly fetched instructions are "flushed" (like in solution 2). This solution is meaningful only if the BTA is known before branch outcome
- ***Solution 4*** - *Branch hoisting (delay slots)* → The compiler moves 3 independent instructions that have to be executed in any case immediately after the *branch* instruction into the so-called *branch delay slots*. When such instructions are complete the branch outcome is known and no stalls are required

Example

Add r2,r1,r3 *Or r4, r1, r5* *Lw r6, 100(r1)* *Bnz r10, #128*	*Bnz r10, 128* *Add r2,r1,r3* *Or r4, r1, r5* *Lw r6, 100(r1)*
Without *bnz* hoisting: stalls necessary.	*bnz* hoisted: stalls not necessary.

Reducing control hazard time penalties



- Although branch delays slots are usually longer than 1 instruction cycle (e.g., 3), in practice only 1 delay slot is filled easily through software (compiler-level) techniques
- To reduce the control hazard time penalty, we can shift both the decision about taken or not taken and the BTA calculation in the ID stage → This requires an additional adder, but it decreases the possible stalls from 3 to 1
- This is not a general rule. In deeply pipelined processors, the control hazard penalties could be very large in terms of wasted clock cycles and can not be solved so easily

Performance of pipelines with stalls

- Structural, data and control hazard may cause stalls in the pipeline. In general, stalls can greatly deteriorate processor performance

$$Speedup = \frac{\overline{CPI}_{unpipelined}}{\overline{CPI}_{pipelined}} * \frac{Tclk_{unpipelined}}{Tclk_{pipelined}}$$

- As the ideal CPI of a pipeline processor is 1, by referring to $\overline{N}_{stalls}$ as the average number of stalls per instruction, we have that:

$$\overline{CPI}_{pipelined} = Ideal_CPI + \overline{N}_{stalls} = 1 + \overline{N}_{stalls}$$

- If we ignore the pipeline overhead, then: $Tclk_{unpipelined} \simeq Tclk_{pipelined}$
- If all instructions take the same number of cycles: $\overline{CPI}_{unpipelined} = pipeline_depth$

$$Speedup = \frac{\overline{CPI}_{unpipelined}}{1 + \overline{N}_{stalls}} = \frac{pipeline_depth}{1 + \overline{N}_{stalls}}$$

In an ideal pipeline without stalls: $Speedup = pipeline_depth$

Computing average stalls

Branch scheme	Penalty unconditional	Penalty untaken	Penalty taken
Flush pipeline	2	3	3
Predicted taken	2	3	2
Predicted untaken	2	0	3

Must still compute the BTA!

Unconditional branch	4%
Conditional branch, untaken	6%
Conditional branch, taken	10%

	Additions to the CPI from branch costs			
Branch scheme	Unconditional branches	Untaken conditional branches	Taken conditional branches	All branches
Frequency of event	4%	6%	10%	20%
Stall pipeline	0.08	0.18	0.30	0.56
Predicted taken	0.08	0.18	0.20	0.46
Predicted untaken	0.08	0.00	0.30	0.38

Speedup computation

- Under the previous assumptions, the speedup for the different solutions can be computed as

$$Speedup = \frac{\overline{CPI}_{unpipelined}}{1 + \overline{N}_{stalls}} = \frac{pipeline_depth}{1 + \overline{N}_{stalls}}$$

  - *pipeline_depth* = 5

  - *Nstalls* – stall pipeline = 0.56
  - *Nstalls* – predict taken = 0.46
  - *Nstalls* – predict untaken = 0.38

  - *Speedup_ideal* = 5 / (1 + 0) = 5
  - *Speedup_stall* = 5 / (1 + 0.56) = 3.21
  - *Speedup_taken* = 5 / (1 + 0.46) = 3.42
  - *Speedup_untaken* = 5 / (1 + 0.38) = 3.62

Branch penalties seriously affect the performance of the pipeline!

EXCEPTIONS

Exceptions - 1

- The term *exceptions* cover all the possible exceptional situations whereby the normal execution order of instructions is changed.
- Many differences exist between various CPUs

Some sources of exception

1. Conditions arising from instruction execution (e.g., overflow, FP anomaly)
2. External interrupts (e.g., I/O device request)
3. System calls (traps)
4. Memory-related exceptions (e.g., cache misses, page fault, memory protection violation, misaligned memory access)
5. Undefined Opcode
6. Hardware malfunctions and power failures

- Consider that not all CPUs are designed to support *all* above causes
- Some further exceptions (e.g., breakpoints, or tracing instruction execution) are considered only during application debug phase

Exception handling

- Non-pipelined architectures allow simple exception management:
  1. Exception is associated with the *only* instruction presently being executed
  2. Exceptions are managed by appropriate routines (e.g., Exception Service Routine or ESR)
  3. When an exception is detected, the CU looks into a vector table. Each row of this table contains the memory address of the corresponding ESR
  4. The ESR saves the PC of the excepting instruction, runs the instruction to handle the corresponding exception and finally returns to the original program if the exception requires resuming

Exception handling

- In pipelined CPUs, the situation is much more involved because *concurrent instructions* are *active* in the pipeline. Two main problems:
  1. Recognizing the exception (i.e., associating it with the correct instruction)
  2. Saving the correct machine state
- Different kinds of exceptions may occur in different points of the pipeline, thus requiring a different handling procedure depending on the properties described above
- *Most difficult scenario*: handling exceptions within instructions that have to be resumed → it requires a *re-startable* pipeline (e.g., a page fault)

Some exception examples in a pipeline

1. *Exceptions arising from instruction execution* (e.g., overflow): usually raised during ***stage EX*** (i.e., within an instruction). Generally user-maskable and they require a resume
2. *External (I/O) interrupts*: Asynchronous, requiring a resume. The ISR are usually served when the interrupted instruction is in the ***WB stage*** (i.e., between subsequent instructions)
3. *Cache miss*: actually, only *data cache miss* are considered exceptions (instruction cache misses are directly managed by hardware, which stalls the CPU before IF phase). These exceptions (like page faults) are detected in ***MEM stage,*** and non-maskable
4. *Non-existing OP code*: may be due to a compiler error, to a memory fault but also, in CPUs belonging to "families", to instructions invoking functional units not present in the particular device used. This exception is raised in the ***ID stage;*** it is non-maskable and leads to program termination

Examples of exceptions in pipeline

Pipeline stage	Problem exceptions occurring
IF	Page fault on instruction fetch; misaligned memory access; memory protection violation
ID	Undefined or illegal opcode
EX	Arithmetic exception
MEM	Page fault on data fetch; misaligned memory access; memory protection violation
WB	None

LD	IF	ID	EX	MEM	WB	
DADD		IF	ID	EX	MEM	WB

- Exceptions may occur **simultaneously** in different stages for different instructions
  - LD raises a page fault in the MEM stage
  - DADD raises an arithmetic exception in the EX stage
- Exceptions may also occur **out of order**
  - LD raises a page fault in the MEM stage
  - DADD raises a page fault in the IF stage
  - The exception for DADD occurs in time before that of LD, even though LD comes first
- Exception vector
  - Exceptions are not handled immediately but posted in a field carried along the pipeline
  - Meanwhile all writes of the instruction and those that follow are disabled
  - Exception handled only when entering in the WB stage

Precise exception handling in a pipeline

Recognize the exception as well as the condition that led to the exception. Post all exceptions in a status vector which is carried along the pipeline.

1. *Turn off both register and memory writes* for all the instructions in the pipeline following the faulting one, but not those before the faulting instruction
   - All instructions ***preceding the one that caused exception*** are normally brought to completion and allowed to modify the machine state (R.F., memory etc)
   - All instructions ***following the one that caused exception*** are ***unexecuted***: they are not allowed to modify the processor state;
2. *When an instruction enters WB, the exception status vector is checked*. If any exceptions are posted, they are handled in the order in which they would occur in time.
3. After the exception-handling routine receives control, it saves the PC of the faulting instruction. This value will be used to return from the exception later on

ADVANCED PIPELINING

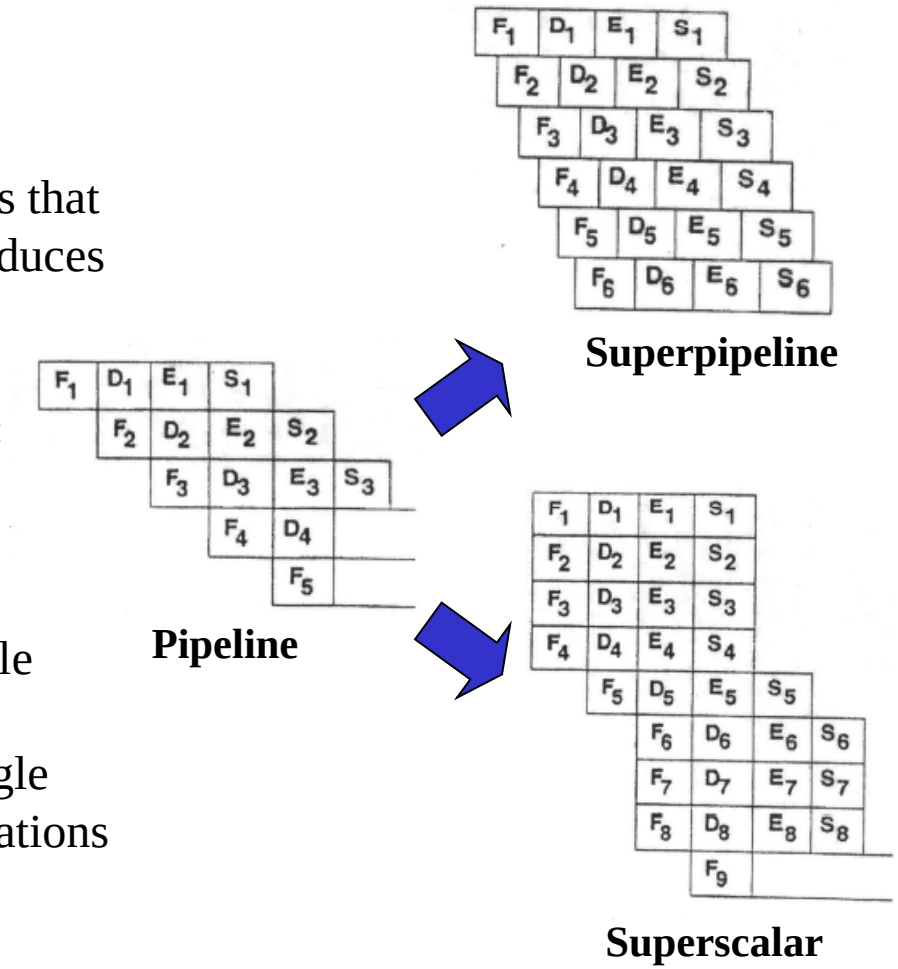
Increasing hardware parallelism

- Besides making the clock cycle shorter, last generation processors tends to improve CPU performance by increasing the hardware parallelism.
- This increases the ideal number of instructions that can be executed in a single clock cycle and reduces the probability of structural hazards.

Usually, two complementary approaches:

- **Superpipelining**: A superpipelined processor has a pipeline where each logical step (IF, ID, MEM, WB) is subdivided into simpler multiple pipeline stages → smaller T_{CLK}
- **Superscalar**: superscalar architecture is a single processor that can execute several scalar operations in parallel

Today, both approaches are used together



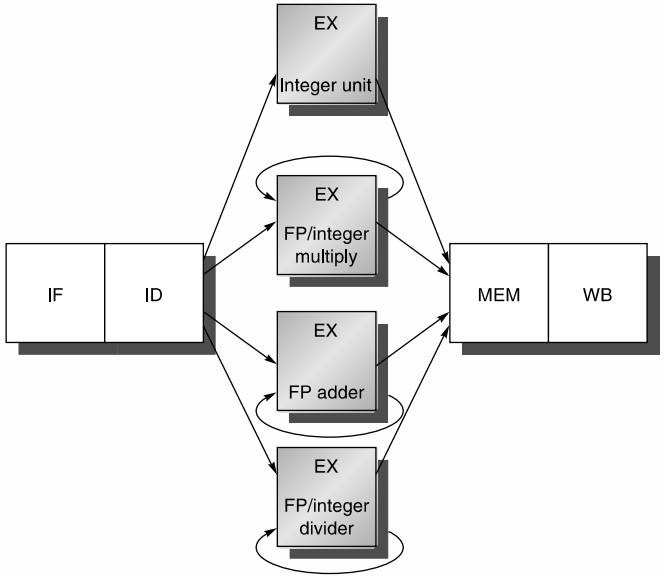
Superpipelining

- In principle, T_{CLK} is inversely proportional to number of stages, given a total instruction latency → the simpler the stage the lower T_{CLK}
- However, reduction of stage complexity is limited by four factors:
  1. *Imbalance problems*: Imbalance between stages reduces performance because the clock period can not be shorter than the time needed for the slowest pipeline stage
  2. *Pipeline overhead time*: It results from the combination of two contributions
     - **Pipeline register delay**: this is due to setup and propagation time through the register. This is constant and related to technology
     - **Clock skew**: worst-case delay between active clock signal edges in two points of a digital system (e.g., between any two pipeline registers)

     The sum of these two contributions represents lower bound to clock cycle (no time left to perform useful work)
  3. *Growing data dependences*: this leads to a higher number of bubbles in a deeper pipeline – performance loss due to data hazards increases
  4. *Longer control hazards*: imply slower branches in case of wrong predictions (i.e., wrongly chosen branches lead to flush a larger *number of stages*)
- *Summing up*: cycle time is shorter but the number of cycles required by a given program becomes larger → usually 12-15 stages in current architectures

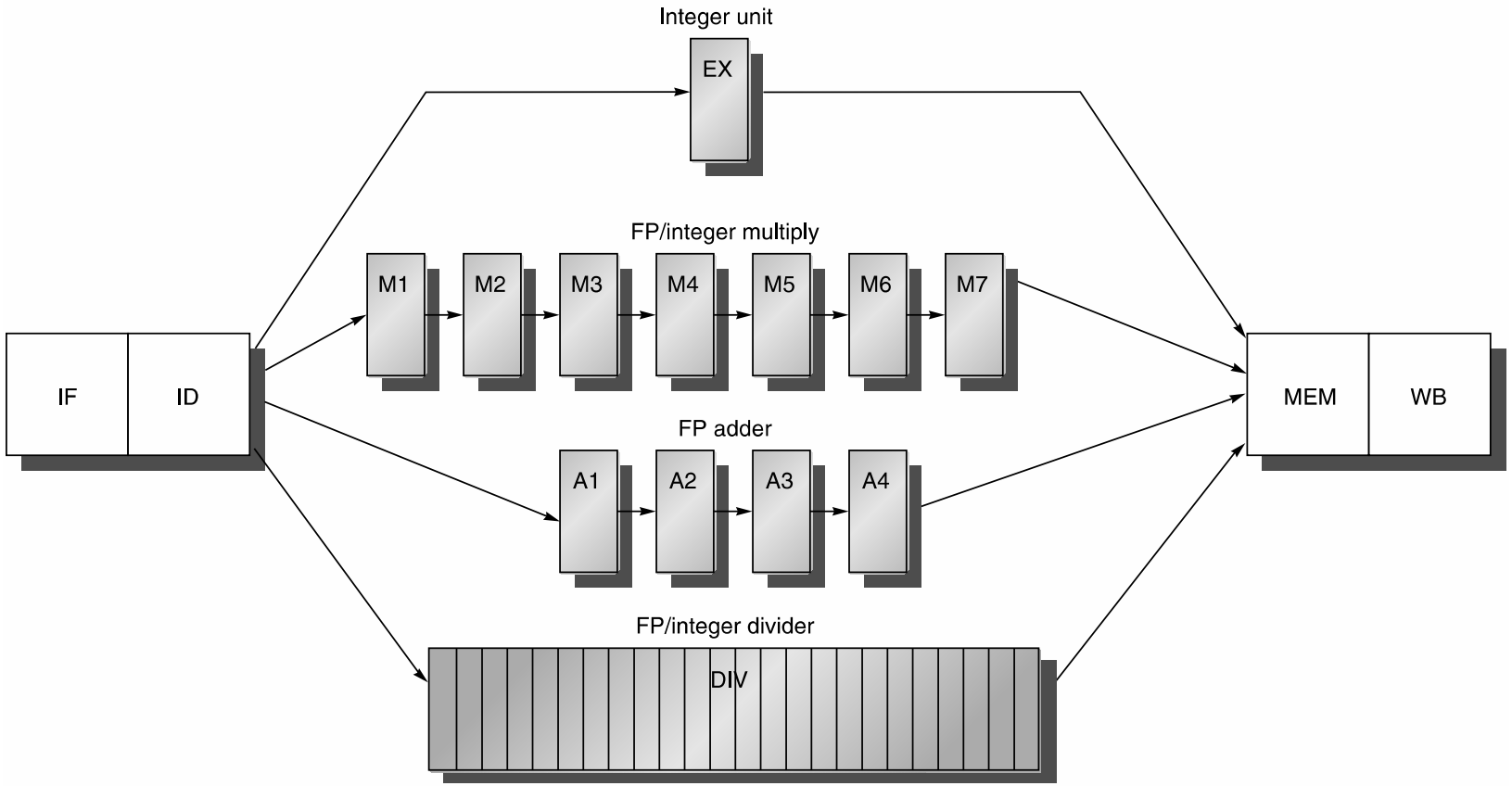
Handling multicycle operations

- Certain operations, such as floating point, may require multiple cycles to complete
  - Impractical to have them complete in one cycle
- The EX stage will be repeated as many times as necessary
- Multiple functional units (EX stages) may be used for different operations
  - Floating point add, multiply, divide, etc.
- A stall occurs if
  - There is a structural hazard (next instruction wants the same functional unit)
  - There is a data hazard



Handling multicycle operations

- Pipelined functional units allow for several outstanding operations



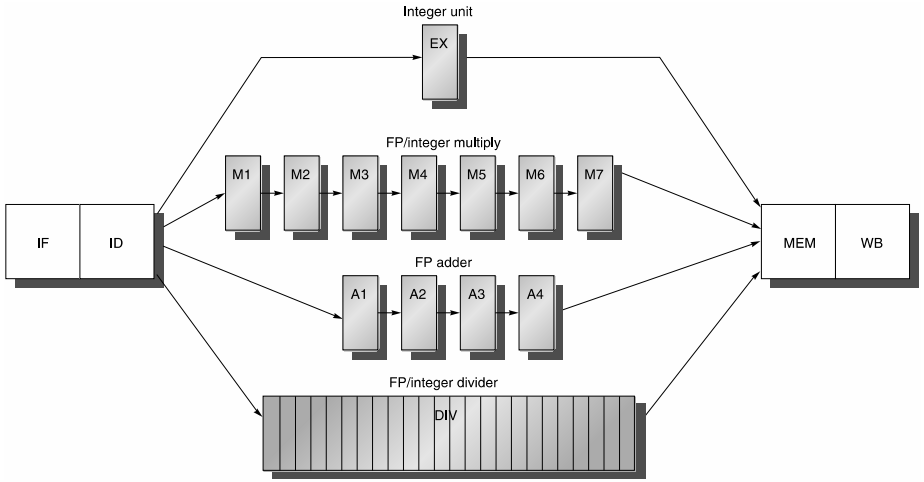
Handling hazards

- Instructions may complete in a different order than they were issued
- WAW hazards are now possible, since instructions no longer reach WB in order
- WAR are still not possible, since registers are still read in ID, and instructions go through ID in order (instruction i + 1 still can't write result before instruction i has read its operands!)
- Stalls due to RAW will be more frequent, since instructions have longer latency
  - Must wait for the instructions to complete

RAW hazards

- Instructions stalls for long periods awaiting the results from the floating point operations

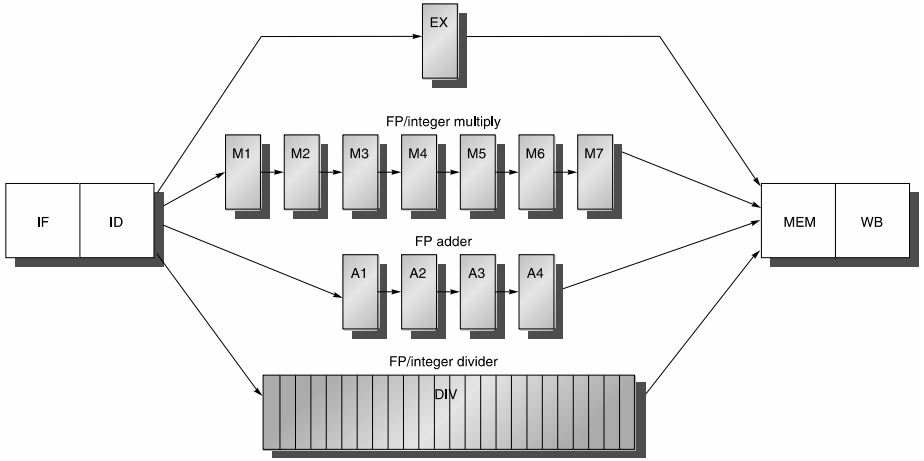
	Clock cycle number																
Instruction	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**	**16**	**17**
`L.D F4,0(R2)`	IF	ID	EX	MEM	WB												
`MUL.D F0,F4,F6`		IF	ID	stall	M1	M2	M3	M4	M5	M6	M7	MEM	WB				
`ADD.D F2,F0,F8`			IF	stall	ID	stall	stall	stall	stall	stall	stall	A1	A2	A3	A4	MEM	WB
`S.D F2,0(R2)`					IF	stall	stall	stall	stall	stall	stall	ID	EX	stall	stall	stall	MEM



Structural hazards

- Instructions reach MEM and WB simultaneously!

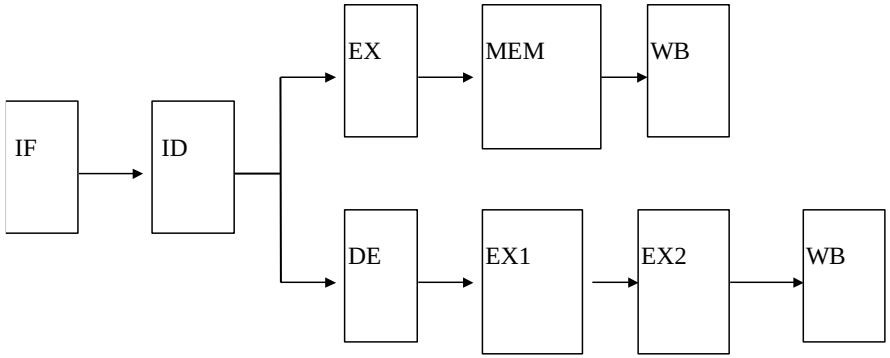
	Clock cycle number										
Instruction	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**
`MUL.D F0,F4,F6`	IF	ID	M1	M2	M3	M4	M5	M6	M7	MEM	WB
`...`		IF	ID	EX	MEM	WB					
`...`			IF	ID	EX	MEM	WB				
`ADD.D F2,F4,F6`				IF	ID	A1	A2	A3	A4	MEM	WB
`...`					IF	ID	EX	MEM	WB		
`...`						IF	ID	EX	MEM	WB	
`L.D  F2,0(R2)`							IF	ID	EX	MEM	WB



Superscalar architectures

- In order to improve performance further, two possible enhancements are:
  1. Using different ***dedicated pipelines***, implemented on the basis of the classes of instructions that are more frequently used on a given architecture

Example



  2. Using ***multiple identical pipelines***, i.e., multiple copies of a given physical pipeline.

- A multiplicity of pipelines requires not only multiple execution units but also:
  – The capacity of ***decoding*** and ***executing*** more than one instruction at a time
  – The capacity of squeezing more parallelism out of the code. This is limited by the number of possible hazards

Further problems…

- Pipelining optimization and exception handling simplification were among the main reasons that caused CPU architectures to shift from CISC to RISC
- In superscalar architectures you need more advanced techniques to extract the parallelism out of the code
- In this case *branch prediction* and *dynamic scheduling* techniques managed at run-time by additional hardware are used. Prediction implies a *speculation* on comparison outcomes → ***speculative execution*** (see next lectures)
- Also, as we have multiple pipelines, the execution times of different instructions are different and maintaining precise exceptions is much more complicated
- In this case *imprecise exceptions* (i.e., related to a status of the processor that is different from the status that the CPU would have if exceptions occurred in fetching order) may happen